

Workbook · อายุ 12–15 ปี · เริ่มจากศูนย์

นักสร้างแอปรุ่นเยาว์

สร้างแอปจัดการใบแจ้งหนี้ ลูกค้า และการชำระเงิน
ทีละขั้นตอน ตั้งแต่ไม่เคยเขียนโค้ดเลย

HTML · CSS · JavaScript · IndexedDB · PWA · Cloud Sync · Testing · Git

ชื่อนักสร้างแอป:

.....

วันที่เริ่มเรียน:

.....

ยินดีต้อนรับ!

สมุดเล่มนี้จะพาคุณสร้าง แอปเว็บจัดการธุรกิจขนาดเล็ก ที่ออกแบบเจ๋งๆ จัดการลูกค้าและสินค้า บันทึกการชำระเงิน ทำรายงาน และใช้งานบนมือถือได้ แม้ไม่มีอินเทอร์เน็ต เหมือนแอปที่ธุรกิจจริงใช้กัน โดยไม่ต้องมีพื้นฐานการเขียนโปรแกรมมาก่อน

วิธีใช้สมุดเล่มนี้

- เรียน ทีละบท บทละ 1–2 ชั่วโมง แนะนำสัปดาห์ละ 2 บท
- พิมพ์โค้ดเอง อย่า copy-paste เพราะนี่จะจำได้ สมอจะเข้าใจ
- ทดสอบทุกครั้งที่เพิ่มโค้ดนิดเดียว ถ้าพังจะรู้ทันทีว่าพังตรงไหน
- ติดแล้วไม่ต้องกลัว! บั๊ก (bug) คือเรื่องปกติของนักเขียนโปรแกรมทุกคน ดูวิธีแก้ในบทที่ 25 ภาคผนวก ค และ ผังวินิจฉัยปัญหาในภาคผนวก จ
- ใช้ ข้อมูลสมมติ เท่านั้น ห้ามใช้ชื่อ/เบอร์โทร/ข้อมูลจริงของคนอื่นตอนฝึก

สัญลักษณ์ในสมุด

เปรียบเทียบให้เห็นภาพ

อธิบายเรื่องยากด้วยของใกล้ตัว

เกร็ดความรู้

เคล็ดลับและข้อมูลน่ารู้

ระวัง!

ข้อผิดพลาดที่คนส่วนใหญ่เจอ

ลองทำ

กิจกรรมลงมือทำ ต้องทำทุกข้อ

ภารกิจท้าทาย

ต่อยอดเอง ไม่มีเฉลยตายตัว

เกี่ยวกับ "แอปจริง" ที่อ้างถึง

ตลอดเล่มจะมีกล่อง "ดูไฟล์เหล่านี้ในแอปจริงประกอบ" ซึ่งชี้ไปยังไฟล์ในโปรเจกต์เว็บแอปจัดการใบแจ้งหนี้ตัวเต็ม (โพลเดอร์ web app) ที่เป็นเป้าหมายปลายทางของเรา แนวทางคือ สร้างเวอร์ชันเล็กของตัวเองก่อน แล้วค่อยเปิดแอปจริงดูว่ามีอาชีพทำต่างจากเราอย่างไร ไม่ต้องกังวลถ้าอ่านโค้ดจริงยังไม่เข้าใจทั้งหมด เป้าหมายคือให้เห็นภาพใหญ่

แผนที่การเดินทาง (26 บท · 6 ด้าน)

ฝึกข้อสุดท้ายเมื่อจบแต่ละบท

บท	หัวข้อ	ด้าน	เวลา	จบแล้ว
1	เว็บแอปคืออะไร และเตรียมโต๊ะทำงาน	ด้าน 1	45–60 นาที	☐
2	HTML เบื้องต้น: หน้าเว็บหน้าแรก	ด้าน 1	60–90 นาที	☐
3	HTML φόρμและตาราง: หน้าตาของใบแจ้งหนี้	ด้าน 1	60–90 นาที	☐
4	CSS เบื้องต้น: สี ขนาด และกล่อง	ด้าน 2	60–90 นาที	☐
5	CSS จัดเลย์เอาต์: Flexbox, Grid และรองรับมือถือ	ด้าน 2	60–90 นาที	☐
6	JavaScript เบื้องต้น: ตัวแปรและฟังก์ชัน	ด้าน 3	60–90 นาที	☐
7	เงื่อนไข วนซ้ำ อาร์เรย์ และอ็อบเจกต์	ด้าน 3	60–90 นาที	☐
8	DOM และ Event: ให้นุ่นทำงาน	ด้าน 3	60–90 นาที	☐
9	โปรเจกต์ย่อย 1: เครื่องคิดเลขใบแจ้งหนี้ (VAT 7%)	ด้าน 3	90–120 นาที	☐
10	หลายหน้า หลายไฟล์: แยกโค้ดให้เป็นระเบียบ	ด้าน 4	60–90 นาที	☐
11	เก็บข้อมูลแบบง่าย: localStorage	ด้าน 4	60–90 นาที	☐
12	ฐานข้อมูลในเบราว์เซอร์: IndexedDB	ด้าน 4	90–120 นาที	☐
13	หน้าจัดการลูกค้า: เพิ่ม แก้ไข ลบ ค้นหา (CRUD)	ด้าน 4	90–120 นาที	☐
14	สินค้าและราคา: ราคามาตรฐาน vs ราคาเฉพาะลูกค้า	ด้าน 4	60–90 นาที	☐
15	สร้างใบแจ้งหนี้: เลขที่เอกสารและกันบันทึกซ้ำ	ด้าน 4	120 นาที	☐
16	รายการใบแจ้งหนี้ ตัวกรอง และการพิมพ์	ด้าน 4	60–90 นาที	☐
17	ระบบชำระเงิน: ยอดค้าง เศษสตางค์ และสถานะ	ด้าน 5	120 นาที	☐
18	แดชบอร์ดและรายงาน: เปลี่ยนข้อมูลเป็นภาพ	ด้าน 5	60–90 นาที	☐
19	นำเข้าและส่งออกไฟล์: CSV และ Excel	ด้าน 5	60–90 นาที	☐
20	ล็อกอินและสิทธิ์ผู้ใช้	ด้าน 6	60–90 นาที	☐
21	PWA: ติดตั้งแอปบนมือถือและใช้ออฟไลน์	ด้าน 6	60–90 นาที	☐
22	Cloud Sync: ให้หลายเครื่องใช้ข้อมูลเดียวกัน	ด้าน 6	120 นาที	☐
23	สำรองและกู้คืนข้อมูล	ด้าน 6	60–90 นาที	☐
24	ทดสอบโปรแกรม: ไฟล์ test-*.js	ด้าน 6	60–90 นาที	☐
25	Git เก็บประวัติโค้ด และวิธีแก้บั๊ก	ด้าน 6	60–90 นาที	☐
26	ก้าวต่อไป: แอปเดสก์ท็อป (Tauri) และโครงสร้างโปรเจกต์ใหญ่	ด้าน 6	60 นาที	☐

ด้าน 1

รู้จักเว็บและ HTML

เป้าหมาย: เรียนรู้ว่าเว็บทำงานอย่างไร และเขียนหน้าเว็บที่มีฟอร์มกับตารางเป็นครั้งแรก

สิ่งที่จะสร้าง: หน้าเว็บใบแจ้งหนี้แบบนิ่ง (ยังไม่ทำงาน)

บทในด้านนี้

- บทที่ 1: เว็บแอปคืออะไร และเตรียมโต๊ะทำงาน
- บทที่ 2: HTML เบื้องต้น: หน้าเว็บหน้าแรก
- บทที่ 3: HTML ฟอร์มและตาราง: หน้าตาของใบแจ้งหนี้

เว็บแอปคืออะไร และเตรียมโต๊ะทำงาน

เวลาที่แนะนำ: 45–60 นาที

เป้าหมาย: เข้าใจว่าเว็บทำงานอย่างไร ติดตั้งเครื่องมือ และสร้างโพลเดอร์โปรเจกต์ของเรา

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `index.html` `style.css`

ลองนึกถึงร้านอาหาร: เรา (ผู้ใช้) สั่งอาหาร, พนักงาน (browser) รับออเดอร์และยกอาหารมาให้, ครั้ว (server) ทำอาหาร ส่วนแอปที่เราจะสร้างเป็นแบบพิเศษ คือ ครั้วอยู่ในมือถือ/คอมของเราเอง ข้อมูลเก็บในเครื่อง ใช้งานได้แม้ไม่มีอินเทอร์เน็ต (เรียกว่า *offline-first*)

เปรียบเทียบให้เห็นภาพ

เว็บเพจมี 3 ส่วนเหมือนร่างกาย: HTML = โครงกระดูก (มีอะไรบ้าง), CSS = เสื้อผ้าและหน้าตา (สวยอย่างไร), JavaScript = สมองและกล้ามเนื้อ (ทำอะไรได้บ้าง)

เครื่องมือที่ต้องมี (ฟรีทั้งหมด)

เครื่องมือ	ใช้ทำอะไร	หมายเหตุ
Google Chrome	เปิดดูและทดสอบแอป	ใช้ปุ่ม F12 เปิด DevTools
Visual Studio Code	โปรแกรมเขียนโค้ด	ติดตั้งส่วนเสริม Live Server ให้เปิดเว็บอัตโนมัติ
Node.js (ติดตั้งภายหลังได้)	รันไฟล์เทสต์ในบทที่ 24	
Git (ติดตั้งภายหลังได้)	เก็บประวัติการแก้ไขโค้ด	ใช้ในบทที่ 25

สร้างโครงสร้างโพลเดอร์

โครงสร้างเริ่มต้น

```
my-invoice-app/  
  index.html    <- หน้าแรก  
  style.css     <- หน้าตา  
  app.js        <- สมอง
```

เกร็ดความรู้

แอปจริงของเรามีไฟล์เป็นสิบ ๆ ไฟล์ แต่ทุกอย่างเริ่มจาก 3 ไฟล์นี้ เราจะค่อย ๆ เพิ่มทีละไฟล์ตลอดคอร์ส

ลองทำ (ทำตามทีละข้อ)

- เปิด Chrome แล้วลองกด F12 สังเกตเห็น Elements, Console, Network, Application (เรายังไม่ต้องเข้าใจ แค่ว่ามีอยู่)
- ติดตั้ง VS Code และส่วนเสริม Live Server
- สร้างโพลเดอร์ `my-invoice-app` และไฟล์ว่าง 3 ไฟล์ตามภาพ

ภารกิจท้าทาย

เปิดเว็บไซต์โปรดของคุณ คลิกขวา เลือก **View page source** แล้วหาคำว่า `<html>` ให้เจอ

คำถามทบทวน

HTML, CSS และ JavaScript ต่างกันอย่างไร (อธิบายสั้น ๆ)?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

HTML เบื้องต้น: หน้าเว็บหน้าแรก

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: เขียนหน้าเว็บแรกด้วยแท็กพื้นฐาน เปิดดูใน Chrome ได้

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `index.html` `help.html`

HTML ประกอบด้วย **แท็ก (tag)** ที่ห่อเนื้อหาไว้ เช่น `<h1>หัวข้อ</h1>` แท็กส่วนใหญ่มี **เปิด** และ **ปิด** (มีเครื่องหมาย `/`)

index.html

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>ร้านของฉัน</title>
</head>
<body>
  <h1>ใบแจ้งหนี้ร้านของฉัน</h1>
  <p>ยินดีต้อนรับ! นี่คือหน้าเว็บแรกของฉัน</p>
  <button>กดฉันสิ</button>
</body>
</html>
```

แท็ก	หน้าที่
<code><h1> – <h6></code>	หัวข้อ (ใหญ่ไปเล็ก)
<code><p></code>	ย่อหน้า
<code></code>	ลิงก์ไปหน้าอื่น
<code></code>	รูปภาพ
<code></code>	รายการจุดนำหน้า
<code><button></code>	ปุ่มกด
<code><div></code>	กล่องว่างไว้จัดกลุ่ม

ระวัง!

ภาษาไทยจะแสดงเพี้ยนถ้าลืม `<meta charset="UTF-8">` และต้องบันทึกไฟล์เป็น UTF-8 เสมอ

head vs body: `<head>` คือข้อมูลหลังบ้านที่ผู้ใช้ไม่เห็น (ชื่อแท็บ ฟอนต์ ไฟล์ CSS) ส่วน `<body>` คือทุกอย่างที่ผู้ใช้เห็นบนหน้าจอ

ลองทำ (ทำตามทีละข้อ)

1. พิมพ์โค้ดข้างบนลงใน index.html (พิมพ์เอง อย่า copy จะจำได้ดีกว่า) แล้วคลิกขวาเลือก Open with Live Server
2. เปลี่ยนข้อความเป็นชื่อร้านของคุณ แล้วกดบันทึก (Ctrl+S) ดูว่าหน้าเว็บเปลี่ยนเองไหม
3. เพิ่ม `<a href="https://www.google.com">ไป Google</a>` และรูปภาพ 1 รูป

ภารกิจท้าทาย

ทำหน้า "เมนูร้านค้า" ที่มีหัวข้อ รายการสินค้า 5 อย่างด้วย `<ul>` และปุ่ม "สั่งซื้อ"

คำถามทบทวน

ถ้าลืมแท็กปิด เช่น ลืม `</p>` จะเกิดอะไรขึ้น?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

HTML ฟอर्मและตาราง: หน้าตาของใบแจ้งหนี้

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: สร้างฟอร์มกรอกรายการสินค้าและตารางแสดงผล ซึ่งเป็นหัวใจของแอปใบแจ้งหนี้

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `invoice-create.html` `customers.html` `products.html`

แอปธุรกิจแทบทุกหน้ามีสองอย่าง คือ **ฟอร์ม** (รับข้อมูลจากผู้ใช้) และ **ตาราง** (แสดงข้อมูลเป็นแถว) ลองดูหน้า `invoice-create.html` และ `customers.html` ของแอปจริง จะเห็นว่าเป็นฟอร์มกับตารางเต็มไปหมด

ส่วนของฟอร์มและตาราง

```
<form id="item-form">
  <label>ชื่อสินค้า
    <input id="name-input" type="text" required>
  </label>
  <label>จำนวน
    <input id="qty-input" type="number" min="1" value="1">
  </label>
  <label>ราคา (บาท)
    <input id="price-input" type="number" min="0" step="0.01">
  </label>
  <button type="submit">เพิ่มรายการ</button>
</form>

<table>
  <thead>
    <tr><th>สินค้า</th><th>จำนวน</th><th>ราคา</th><th>รวม</th></tr>
  </thead>
  <tbody id="items"></tbody>
</table>

<p>ยอดรวม: <strong id="total">0.00</strong> บาท</p>
```

เกร็ดความรู้

id คือชื่อเฉพาะของแต่ละองค์ประกอบ ไม่ซ้ำกันในหน้าเดียว JavaScript จะใช้ **id** เพื่อ "เรียกหา" องค์ประกอบนั้นในบทที่ 8

ชนิด input	ใช้กับ
<code>type="text"</code>	ชื่อ ที่อยู่
<code>type="number"</code>	จำนวน ราคา
<code>type="date"</code>	วันที่ออกใบแจ้งหนี้
<code><select></code>	เลือกจากรายการ เช่น ชื่อลูกค้า
<code>type="checkbox"</code>	ติ๊กถูก/ไม่ถูก

ลองทำ (ทำตามทีละข้อ)

1. ใส่โค้ดฟอร์มและตารางลงใน `<body>` ของ `index.html`
2. เพิ่มช่องเลือกวันที่ (`type=date`) และ `<select>` ที่มีตัวเลือกชื่อลูกค้า 3 คน
3. กดปุ่มดูว่าอะไรเกิดขึ้น (ตอนนี้ยังไม่เกิดอะไร เพราะยังไม่มี JavaScript เราจะแก้ในบทที่ 8)

ภารกิจท้าทาย

ออกแบบฟอร์ม "เพิ่มลูกค้า" ที่มีช่องชื่อ เบอร์โทร ที่อยู่ (ใช้ `<textarea>`) และปุ่มบันทึก

คำถามทบทวน

ทำไม `input` แต่ละช่องควรมี `id`?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ด้าน 2

ตกแต่งด้วย CSS

เป้าหมาย: ทำให้หน้าเว็บสวยและใช้ได้ดีทั้งบนคอมพิวเตอร์และมือถือ

สิ่งที่จะสร้าง: หน้าตามืออาชีพ ใช้ซ้ำได้ทุกหน้า

บทในตอนนี้

- บทที่ 4: CSS เบื้องต้น: สี ขนาด และกล่อง
- บทที่ 5: CSS จัดเลย์เออต์: Flexbox, Grid และรองรับมือถือ

4

ด้าน 2 : ตกแต่งด้วย CSS

CSS เบื้องต้น: สี ขนาด และกล่อง

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ตกแต่งหน้าเว็บด้วยสี ตัวอักษร และเข้าใจ box model

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `style.css`

CSS คือชุดกฎที่บอกว่า "องค์ประกอบแบบไหน ให้นำหน้าตาอย่างไร" รูปแบบคือ ตัวเลือก { คุณสมบัติ: ค่า; }

style.css

```
:root { --main:#2563eb; --bg:#f1f5f9; } /* ตัวแปรสี ใช้ซ้ำได้ */
* { box-sizing: border-box; }
body {
  font-family: "Sarabun", sans-serif;
  background: var(--bg);
  margin: 0; padding: 16px;
}
h1 { color: var(--main); }
button {
  background: var(--main); color: white;
  border: 0; padding: 10px 16px;
  border-radius: 8px; font-size: 16px; cursor: pointer;
}
button:hover { opacity: .85; }
.card {
  background: white; padding: 16px;
  border-radius: 12px; box-shadow: 0 2px 6px rgba(0,0,0,.1);
}
```

อย่าลืมเชื่อมไฟล์ CSS ใน `<head>` ของ HTML:`<link rel="stylesheet" href="style.css">`

Box Model: ทุกอย่างคือกล่อง

เปรียบเทียบให้เห็นภาพ

องค์ประกอบ HTML เหมือน กรอบรูป: ตัวรูป = **content**, ระยะระหว่างรูปกับกรอบ = **padding**, ขอบกรอบ = **border**, ระยะห่างจากกรอบอื่น = **margin**

เลือกด้วย	ตัวอย่าง	ความหมาย
ชื่อแท็ก	<code>button { }</code>	ทุกปุ่ม
class (จุด)	<code>.card { }</code>	ทุกอย่างที่ใส่ <code>class="card"</code>
id (ขีด #)	<code>#total { }</code>	องค์ประกอบเดียวที่มี <code>id="total"</code>

เกร็ดความรู้

ไฟล์ `style.css` ของแอปจริงใช้ร่วมกันทุกหน้า จึงทำให้ปุ่มและสีเหมือนกันทั้งแอป เปลี่ยนที่เดียวเปลี่ยนทั้งแอป

ลองทำ (ทำตามทีละข้อ)

1. สร้าง style.css แล้วเชื่อมกับ index.html
2. ห่อฟอร์มและตารางด้วย `<div class="card">`
3. เปลี่ยน --main เป็นสีโปรดของคุณ ดูว่าทั้งหน้าเปลี่ยนสีตามไหม

ภารกิจท้าทาย

ทำปุ่ม 2 แบบ คือปุ่มหลักสีน้ำเงิน และปุ่ม "ลบ" สีแดง โดยใช้ class ต่างกัน

คำถามทบทวน

ข้อดีของการใช้ตัวแปร CSS เช่น --main คืออะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

5

ด้าน 2 : ตกแต่งด้วย CSS

CSS จัดเลย์เอาต์: Flexbox, Grid และรองรับมือถือ

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: จัดวางองค์ประกอบเป็นแถว/ตาราง และทำให้หน้าเว็บใช้งานได้ดีบนมือถือ

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `style.css` `nav.js` `dashboard.html`

แอปของเราเป็น PWA ที่ต้องใช้บนมือถือได้ จึงต้องออกแบบแบบ responsive คือปรับตามขนาดจอ

ตัวอย่างเลย์เอาต์

```
/* Flexbox: เรียงของในแถวเดียว */
.nav { display: flex; gap: 8px; background: var(--main); padding: 8px; }
.nav a { color: white; padding: 8px 12px; text-decoration: none; border-radius: 6px; }
.nav a.active { background: rgba(255,255,255,.25); }

/* Grid: แบ่งเป็นช่อง เช่น การ์ดสรุปตัวเลข */
.stats { display: grid; grid-template-columns: repeat(3, 1fr); gap: 12px; }

/* จอแคบกว่า 600px ให้เรียงเป็นคอลัมน์เดียว */
@media (max-width: 600px) {
  .stats { grid-template-columns: 1fr; }
  .nav { flex-wrap: wrap; }
}
```

HTML ที่ใช้คู่กัน

```
<div class="nav">
  <a href="index.html" class="active">หน้าแรก</a>
  <a href="customers.html">ลูกค้า</a>
  <a href="invoices.html">ใบแจ้งหนี้</a>
</div>
<div class="stats">
  <div class="card">ยอดขายเดือนนี้<br><strong>12,500</strong></div>
  <div class="card">ค้างชำระ<br><strong>3,200</strong></div>
  <div class="card">ลูกค้า<br><strong>18</strong></div>
</div>
```

เกร็ดความรู้

ทดสอบมือถือโดยกด F12 แล้วกดไอคอนรูปมือถือ (Toggle device toolbar) ใน Chrome ไม่ต้องมีมือถือจริงก็ได้

ลองทำ (ทำตามทีละข้อ)

1. ทำแถบเมนูด้านบนด้วย Flexbox
2. ทำการ์ดสรุป 3 ใบด้วย Grid
3. ลองย่อ/ขยายหน้าต่างเบราว์เซอร์ ดูว่าการ์ดเปลี่ยนเป็นแนวตั้งเมื่อจอแคบไหม

ภารกิจท้าทาย

เพิ่มปุ่ม "เมนู" ที่ปรากฏเฉพาะบนมือถือ (ซ่อนด้วย display:none บนจอกว้างโดยใช้ media query)

คำถามทบทวน

Flexbox กับ Grid ใช้ต่างกันอย่างไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ด้าน 3

JavaScript ทำให้แอปมีชีวิต

เป้าหมาย: เขียนโปรแกรมให้หน้าเว็บคิดเลข ตอบสนอง และจัดการรายการได้

สิ่งที่ sẽ สร้าง: โปรเจกต์ย่อย 1: เครื่องคิดเลขใบแจ้งหนี้ + VAT

บทในตอนนี้

- บทที่ 6: JavaScript เบื้องต้น: ตัวแปรและฟังก์ชัน
- บทที่ 7: เงื่อนไข วนซ้ำ อาร์เรย์ และอ็อบเจกต์
- บทที่ 8: DOM และ Event: ให้ปุ่มทำงาน
- บทที่ 9: โปรเจกต์ย่อย 1: เครื่องคิดเลขใบแจ้งหนี้ (VAT 7%)

6

ด้าน 3 · JavaScript ทำให้แอปมีชีวิต

JavaScript เบื้องต้น: ตัวแปรและฟังก์ชัน

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: เขียนโปรแกรมแรกด้วย JavaScript ใช้ตัวแปรและฟังก์ชันคำนวณเงินได้

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `utils.js`

JavaScript ให้นำเว็บ "คิดและตอบสนอง" ได้ เริ่มจากฝึกใน Console (กด F12 แล้วเลือกแท็บ Console)

เปรียบเทียบให้เห็นภาพ

ตัวแปร = กล่องมีป้ายชื่อ ใส่ของไว้ได้ (`let` = กล่องที่เปลี่ยนของได้, `const` = กล่องที่ล็อกไว้) ส่วน ฟังก์ชัน = สูตรอาหาร ใส่วัตถุดิบเข้าไป ได้อาหารออกมา

app.js

```
let shopName = "ร้านของฉัน"; // ข้อความ (string)
const VAT_RATE = 0.07;      // ตัวเลข (number) ค่าคงที่
let isOpen = true;           // จริง/เท็จ (boolean)

function addVat(amount) {
  return amount * (1 + VAT_RATE);
}

console.log(shopName);
console.log(addVat(100)); // 107
```

เชื่อมไฟล์ JavaScript ท้าย `<body>` :`<script src="app.js"></script>`

ชนิดข้อมูล	ตัวอย่าง
string (ข้อความ)	"สวัสดี"
number (ตัวเลข)	42, 3.14
boolean	true, false
array (รายการ)	[1, 2, 3]
object (ข้อมูลที่มีหลายช่อง)	{name:"ปากกา", price:10}

ลองทำ (ทำตามทีละข้อ)

- เปิด Console แล้วพิมพ์ `2 + 3 * 4` ดูผล
- ทำฟังก์ชัน `calcTotal(qty, price)` ที่คืนค่า `qty * price`
- ทำฟังก์ชัน `addVat` และ `removeVat` (ถอด VAT ออกจากราคารวม)

ภารกิจท้าทาย

เขียนฟังก์ชัน `discount(amount, percent)` ที่คืนยอดหลังลดราคา แล้วทดสอบกับ 1,000 บาท ลด 15%

คำถามทบทวน

let กับ const ต่างกันอย่างไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

7

ด้าน 3 · JavaScript ทำให้แอปมีชีวิต

เงื่อนไข วงซ้ำ อาร์เรย์ และอ็อบเจกต์

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ตัดสินใจด้วย if วงซ้ำด้วย for และเก็บรายการสินค้าเป็นอาร์เรย์ของอ็อบเจกต์

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `utils.js` `reports.html`ใบแจ้งหนี้หนึ่งใบมีสินค้าหลายรายการ เราจึงเก็บเป็น **array** (รายการ) ของ **object** (สินค้าแต่ละชิ้น)

app.js

```
const items = [
  { name: "ปากกา", qty: 3, price: 10 },
  { name: "สมุด", qty: 2, price: 25.5 },
];

// วงซ้ำด้วย for...of
let subtotal = 0;
for (const it of items) {
  subtotal += it.qty * it.price;
}

// วิธีสั้นกว่า: reduce
const subtotal2 = items.reduce((sum, it) => sum + it.qty * it.price, 0);

// เงื่อนไข
if (subtotal2 > 100) {
  console.log("ลูกค้าได้ส่วนลด 5%");
} else {
  console.log("ไม่มีส่วนลด");
}

// เครื่องมือประจำของ array
const names = items.map(it => it.name); // ["ปากกา", "สมุด"]
const bigOnes = items.filter(it => it.qty >= 3); // เลือกบางชิ้น
const pen = items.find(it => it.name === "ปากกา");
```

เครื่องมือ	ทำอะไร	ใช้ในแอปจริงตรงไหน
<code>map</code>	แปลงทุกชิ้นเป็นอย่างอื่น	สร้างแถวตารางจากข้อมูล
<code>filter</code>	คัดเฉพาะบางชิ้น	ค้นหาลูกค้า กรองใบแจ้งหนี้ที่ยังไม่จ่าย
<code>find</code>	หาชิ้นแรกที่ตรง	หาลูกค้าจาก id
<code>reduce</code>	รวมทั้งหมดเป็นค่าเดียว	ยอดรวม ยอดค้างชำระ
<code>sort</code>	เรียงลำดับ	เรียงใบแจ้งหนี้ตามวันที่

ลองทำ (ทำตามทีละข้อ)

- ใส่ข้อมูลสินค้า 5 รายการ แล้วหายอดรวมทั้งสองวิธี
- ใช้ `filter` หาสินค้าที่ราคามากกว่า 20 บาท
- ใช้ `sort` เรียงสินค้าจากแพงไปถูก: `items.sort((a,b) => b.price - a.price)`

ภารกิจท้าทาย

หาสินค้าที่แพงที่สุดในรายการ โดยใช้ `reduce` หรือ `sort`

คำถามทบทวน

`[1,2,3].map(x => x * 2)` ได้อะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

8

ด้าน 3 · JavaScript ทำให้แอปมีชีวิต

DOM และ Event: ให้ปุ่มทำงาน

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ใช้ JavaScript อ่านค่าจากฟอร์ม เพิ่มแถวในตาราง และตอบสนองเมื่อผู้ใช้กดปุ่มดูไฟล์เหล่านี้ในแอปจริงประกอบ: `invoice-create.html` `customers.html`

DOM คือ "ต้นไม้" ของทุกองค์ประกอบบนหน้าเว็บที่ JavaScript จับและแก้ไขได้ ส่วน Event คือเหตุการณ์ เช่น คลิก พิมพ์ ส่งฟอร์ม

app.js

```
const form = document.getElementById("item-form");
const tbody = document.getElementById("items");
const items = [];

form.addEventListener("submit", (event) => {
  event.preventDefault(); // ไม่ให้หน้ารีโหลด
  const item = {
    name: document.getElementById("name-input").value.trim(),
    qty: Number(document.getElementById("qty-input").value),
    price: Number(document.getElementById("price-input").value),
  };
  if (!item.name || item.qty <= 0) return; // ตรวจสอบข้อมูล
  items.push(item);
  form.reset();
  render();
});

function render() {
  tbody.innerHTML = "";
  let total = 0;
  for (const it of items) {
    const tr = document.createElement("tr");
    for (const text of [it.name, it.qty, it.price, it.qty * it.price]) {
      const td = document.createElement("td");
      td.textContent = text; // ปลอดภัยกว่า innerHTML
      tr.appendChild(td);
    }
    tbody.appendChild(tr);
    total += it.qty * it.price;
  }
  document.getElementById("total").textContent = total.toFixed(2);
}
```

ระวัง!

ถ้าเอาข้อความที่ผู้ใช้พิมพ์ไปใส่ใน `innerHTML` ตรง ๆ คนไม่หวังดีอาจแอบใส่โค้ดอันตรายได้ (เรียกว่า **XSS**) ให้ใช้ `textContent` สำหรับข้อความจากผู้ใช้เสมอ

เกร็ดความรู้

แนวคิดสำคัญ: เก็บข้อมูลไว้ใน array ก่อน แล้ว "วาดหน้าจอใหม่จากข้อมูล" ด้วย `render()` ทุกครั้งที่ข้อมูลเปลี่ยน นี่คือการหลักการเดียวกับที่แอปจริงทุกหน้าใช้

ลองทำ (ทำตามทีละข้อ)

1. เพิ่มโค้ดนี้เข้า app.js แล้วลองเพิ่มสินค้า 3 รายการ
2. กดเพิ่มโดยไม่กรอกชื่อ ดูว่าโปรแกรมกันได้ไหม
3. ใช้ `console.log(items)` ดูข้อมูลใน Console

ภารกิจท้าทาย

เพิ่มปุ่ม "ลบ" ในแต่ละแถว (ใช้: ใช้ `items.splice(index, 1)` แล้วเรียก `render()` ใหม่)

คำถามทบทวน

ทำไมจึงต้องเรียก `event.preventDefault()` ตอน submit?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

โปรเจกต์ย่อย 1: เครื่องคิดเลขใบแจ้งหนี้ (VAT 7%)

เวลาที่แนะนำ: 90–120 นาที

เป้าหมาย: ครอบคลุมความรู้ทั้ง 3 ด้านเป็นแอปเล็ก ๆ ที่ใช้งานได้จริง

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `invoice-create.html` `utils.js`

นี่คือ ชัยชนะครั้งแรก ของเรา แอปเครื่องคิดเลขที่เพิ่มสินค้า ลบสินค้า และคำนวณ VAT อัตโนมัติ ซึ่งเป็นแกนกลางเดียวกับหน้า `invoice-create.html` ของแอปจริง (แค่แอปจริงมีฟีเจอร์เยอะกว่ามาก)

`app.js` (เวอร์ชันสมบูรณ์)

```
const VAT_RATE = 0.07;
let items = [];
const form = document.getElementById("item-form");
const tbody = document.getElementById("items");

function money(n) {
  return n.toLocaleString("th-TH", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
}

function cell(text) {
  const td = document.createElement("td");
  td.textContent = text;
  return td;
}

function render() {
  tbody.innerHTML = "";
  items.forEach((it, i) => {
    const tr = document.createElement("tr");
    tr.append(cell(it.name), cell(it.qty), cell(money(it.price)), cell(money(it.qty * it.price)));
    const td = document.createElement("td");
    const btn = document.createElement("button");
    btn.textContent = "ลบ";
    btn.dataset.index = i;
    td.appendChild(btn);
    tr.appendChild(td);
    tbody.appendChild(tr);
  });
  const subtotal = items.reduce((s, it) => s + it.qty * it.price, 0);
  const vat = subtotal * VAT_RATE;
  document.getElementById("subtotal").textContent = money(subtotal);
  document.getElementById("vat").textContent = money(vat);
  document.getElementById("total").textContent = money(subtotal + vat);
}

form.addEventListener("submit", (e) => {
  e.preventDefault();
  const name = document.getElementById("name-input").value.trim();
  const qty = Number(document.getElementById("qty-input").value);
  const price = Number(document.getElementById("price-input").value);
  if (!name || qty <= 0 || price < 0) return;
  items.push({ name, qty, price });
  form.reset();
  render();
});

tbody.addEventListener("click", (e) => { // ฟังก์ชันที่ tbody ที่เดียว
```

```
if (e.target.dataset.index !== undefined) {
  items.splice(Number(e.target.dataset.index), 1);
  render();
}
});
render();
```

ใน HTML ให้เพิ่มบรรทัดสรุปยอดใต้ตาราง (และเพิ่ม `<th></th>` วาง 1 ช่องสำหรับปุ่มลบ):

```
<p>ราคาก่อน VAT: <span id="subtotal">0.00</span></p>
<p>VAT 7%: <span id="vat">0.00</span></p>
<p>ยอดสุทธิ: <strong id="total">0.00</strong> บาท</p>
```

ทำไมตัวเลขยาวผิดปกติ?

ถ้าลองคำนวณใน Console อาจเห็น `300 * 0.07` ได้ `21.000000000000004` ไม่ต้องตกใจ เราใช้ `toLocaleString` ปิดเป็น 2 ตำแหน่งตอนแสดงผลจึงเห็น 21.00 ปัญหานี้ใช้ได้กับเครื่องคิดเลขเล็ก ๆ แต่ ห้ามใช้กับบัญชีจริง ในบทที่ 17 เราจะเรียนรู้วิธีแก้ที่ถูกต้อง

เป้าหมายตอนจบด้านที่ 3

เพิ่มสินค้าได้ ลบได้ ยอดรวมและ VAT คำนวณถูกต้อง ไฟล์ทั้ง 3 (HTML/CSS/JS) ทำงานร่วมกัน

ลองทำ (ทำตามทีละข้อ)

1. ทำตามโค้ดทีละส่วน ทดสอบทุกครั้งที่เพิ่มโค้ดใหม่ (อย่าพิมพ์ทั้งหมดแล้วค่อยลอง)
2. ทดสอบด้วยตัวเลข: 3 ชิ้น ชิ้นละ 100 บาท ต้องได้ ก่อน VAT 300.00 / VAT 21.00 / รวม 321.00
3. ตกแต่งด้วย CSS ที่เรียนมา

ภารกิจท้าทาย

เพิ่มช่อง ส่วนลด (%) ที่หักก่อนคิด VAT และเพิ่มปุ่ม "ล้างทั้งหมด"

คำถามทบทวน

ทำไมจึงเก็บข้อมูลไว้ใน array แล้วค่อย render() แทนที่จะแก้ตารางตรง ๆ?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ด้าน 4

เก็บข้อมูลและสร้างหน้าหลัก

เป้าหมาย: เก็บข้อมูลถาวร สร้างหน้าลูกค้า สินค้า และสร้างใบแจ้งหนี้จริง

สิ่งที่จะสร้าง: แกนหลักของแอป: ลูกค้า + สินค้า + ใบแจ้งหนี้ (เก็บใน IndexedDB)

บทในด้านนี้

- บทที่ 10: หลายหน้า หลายไฟล์: แยกโค้ดให้เป็นระเบียบ
- บทที่ 11: เก็บข้อมูลแบบง่าย: localStorage
- บทที่ 12: ฐานข้อมูลในเบราว์เซอร์: IndexedDB
- บทที่ 13: หน้าจัดการลูกค้า: เพิ่ม แก้ไข ลบ ค้นหา (CRUD)
- บทที่ 14: สินค้าและราคา: ราคามาตรฐาน vs ราคาเฉพาะลูกค้า
- บทที่ 15: สร้างใบแจ้งหนี้: เลขที่เอกสารและกันบันทึกซ้ำ
- บทที่ 16: รายการใบแจ้งหนี้ ตัวกรอง และการพิมพ์

หลายหน้า หลายไฟล์: แยกโค้ดให้เป็นระเบียบ

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: แยกโค้ดที่ใช้ร่วมกัน (เมนู ฟังก์ชันช่วย) ออกเป็นไฟล์ของตัวเอง และสร้างหลายหน้าดูไฟล์เหล่านี้ในแอปจริงประกอบ: `nav.js` `utils.js` `style.css`

แอปจริงมีหน้าหลายสิบหน้า (customers, invoices, payments, reports ...) ถ้าเราคัดลอกเมนูและฟังก์ชันไปแปะทุกหน้า การแก้ไขแต่ละครั้งจะเหนื่อยมาก วิธีแก้คือ แยกไฟล์ที่ใช้ร่วมกัน แล้วเรียกใช้ในทุกหน้า

ไฟล์	หน้าที่	ในแอปจริง
<code>style.css</code>	หน้าตาที่ใช้ร่วมกัน	<code>style.css</code>
<code>utils.js</code>	ฟังก์ชันช่วย เช่น จัดรูปแบบเงิน วันที่	<code>utils.js</code>
<code>nav.js</code>	สร้างเมนูด้านบนให้ทุกหน้า	<code>nav.js</code>
<code>customers.html</code>	หน้าเฉพาะเรื่อง	<code>customers.html</code> , <code>invoices.html</code> ...

`utils.js`

```
// utils.js
function formatMoney(n) {
  return Number(n).toLocaleString("th-TH", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
}
function formatDate(iso) {
  // "2026-10-07" -> "07/10/2569"
  const d = new Date(iso);
  return d.toLocaleDateString("th-TH");
}
```

`nav.js`

```
// nav.js
const PAGES = [
  { href: "index.html", label: "หน้าแรก" },
  { href: "customers.html", label: "ลูกค้า" },
  { href: "products.html", label: "สินค้า" },
  { href: "invoices.html", label: "ใบแจ้งหนี้" },
  { href: "payments.html", label: "ชำระเงิน" },
  { href: "reports.html", label: "รายงาน" },
];
function renderNav() {
  const nav = document.getElementById("nav");
  const current = location.pathname.split("/").pop() || "index.html";
  for (const p of PAGES) {
    const a = document.createElement("a");
    a.href = p.href;
    a.textContent = p.label;
    if (p.href === current) a.className = "active";
    nav.appendChild(a);
  }
}
renderNav();
```

วิธีเรียกใช้ในทุกหน้า

```
<!-- ท้ายทุกหน้า -->
<div id="nav" class="nav"></div> <!-- วางไว้บนสุดของ body -->
...
<script src="utils.js"></script>
<script src="nav.js"></script>
<script src="app.js"></script>
```

ระวัง!

ลำดับสำคัญ! ไฟล์ที่ถูกเรียกใช้ต้องโหลดก่อนไฟล์ที่เรียกใช้มัน (utils.js ก่อน app.js)

ลองทำ (ทำตามทีละข้อ)

1. แยก formatMoney ไปไว้ใน utils.js แล้วแก้เครื่องหมายบัพที่ 9 ให้เรียกใช้
2. สร้างหน้า customers.html และ invoices.html (ใส่แค่หัวข้อก่อน) แล้วดูเมนูว่าแสดงทุกหน้าไหม
3. เพิ่มหน้าใหม่ 1 หน้าโดยแก้ที่ nav.js ที่เดียว

ภารกิจท้าทาย

ทำให้ nav.js ซ่อนลิงก์บางหน้าได้ เช่น เพิ่ม adminOnly: true ให้หน้า "ตั้งค่า" (เตรียมตัวสำหรับบทที่ 20)

คำถามทบทวน

ข้อดีของการแยก nav.js ออกมาคืออะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

เก็บข้อมูลแบบง่าย: localStorage

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: บันทึกข้อมูลลูกค้าให้อยู่รอดแม้ปิดเบราว์เซอร์

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `settings.js` `utils.js`

ที่ผ่านมา เมื่อรีเฟรชหน้า ข้อมูลหายหมด! เพราะข้อมูลอยู่ใน "ความจำชั่วคราว" ของ JavaScript เราจะใช้ `localStorage` ซึ่งเป็นลิ้นชักเล็ก ๆ ของเบราว์เซอร์เก็บข้อความไว้ถาวร

storage.js

```
function loadCustomers() {
  try {
    return JSON.parse(localStorage.getItem("customers")) || [];
  } catch {
    return []; // ถ้าข้อมูลเสีย ให้เริ่มใหม่ ไม่ให้แอปพัง
  }
}

function saveCustomers(list) {
  localStorage.setItem("customers", JSON.stringify(list));
}

const customers = loadCustomers();
customers.push({ id: crypto.randomUUID(), name: "คุณสมชาย", phone: "081-234-5678" });
saveCustomers(customers);
```

เปรียบเทียบให้เห็นภาพ

`localStorage` เก็บได้เฉพาะ **ข้อความ** เหมือนลิ้นชักที่ใส่ได้แต่กระดาษ ดังนั้นเราต้อง "พับเป็นกระดาษ" ด้วย `JSON.stringify` ก่อนเก็บ และ "กางออก" ด้วย `JSON.parse` ตอนอ่าน

ข้อดี	ข้อจำกัด
ใช้ง่ายมาก โค้ดสั้น	เก็บได้ประมาณ 5 MB เท่านั้น
อยู่ถาวรจนกว่าจะล้างข้อมูลเบราว์เซอร์	เก็บได้แต่ข้อความ ค้นหาข้อมูลเร็ว ๆ ไม่ได้
ไม่ต้องติดตั้งอะไร	ทำงานแบบ "ทีละอย่าง" ถ้าข้อมูลเยอะจะช้า

ดังนั้นสำหรับแอปจริงที่มีลูกค้าและใบแจ้งหนี้เป็นพันรายการ เราต้องการสิ่งที่แข็งแกร่งกว่านี้ ซึ่งคือบดอัดไป

ลองทำ (ทำตามทีละข้อ)

- ทำหน้าลูกค้าแบบง่าย: ฟормเพิ่มชื่อ + รายชื่อด้านล่าง ที่ยังอยู่หลังรีเฟรช
- เปิด F12 แท็บ Application → Local Storage ดูข้อมูลที่เก็บไว้
- ลองลบข้อมูลใน Local Storage ด้วยมือ แล้วดูว่าแอปยังไม่พัง

ภารกิจท้าทาย

ทำให้ฟอร์มกันชื่อลูกค้าซ้ำ (ใช้ `customers.some(c => c.name === newName)`)

คำถามทบทวน

ทำไมต้องครอบ JSON.parse ด้วย try/catch?

(เฉลยอยู่ในภาคผนวก จ)

เช็กलिस्टก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ฐานข้อมูลในเบราว์เซอร์: IndexedDB

เวลาที่แนะนำ: 90–120 นาที

เป้าหมาย: เข้าใจว่าทำไมแอปจึงใช้ IndexedDB และเขียนฟังก์ชันบันทึก/อ่าน/ลบข้อมูลได้

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `idb.js` `db.js`

IndexedDB คือฐานข้อมูลเต็มรูปแบบที่อยู่ในเบราว์เซอร์ เก็บ object ได้ตรง ๆ ไม่ต้องแปลงเป็นข้อความ เก็บได้เป็นร้อยเมกะไบต์ และค้นหาได้เร็ว แอปจริงของเรามีไฟล์ `idb.js` และ `db.js` (ไฟล์ใหญ่ที่สุดไฟล์หนึ่ง) ทำหน้าที่นี้

เปรียบเทียบให้เห็นภาพ

`localStorage` = กระดานโน้ตแปะตู้เย็น, `IndexedDB` = ตูเอกสารจัดหมวดหมู่ มีลิ้นชัก (store) หลายลิ้นชัก เช่น ลิ้นชักลูกค้า ลิ้นชักใบแจ้งหนี้ แต่ละแฟ้มมีเลขที่ (key) ของตัวเอง

`db.js` (เวอร์ชันเด็ก)

```
function openDB() {
  return new Promise((resolve, reject) => {
    const req = indexedDB.open("invoice-app", 1);
    req.onupgradeneeded = () => { // ทำครั้งแรก หรือเมื่อเปลี่ยนเวอร์ชัน
      const db = req.result;
      db.createObjectStore("customers", { keyPath: "id" });
      db.createObjectStore("invoices", { keyPath: "id" });
    };
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

async function dbPut(store, value) { // เพิ่มหรือแก้ไข
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(store, "readwrite");
    tx.objectStore(store).put(value);
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}

async function dbGetAll(store) { // อ่านทั้งหมด
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const req = db.transaction(store).objectStore(store).getAll();
    req.onsuccess = () => resolve(req.result);
    req.onerror = () => reject(req.error);
  });
}

async function dbDelete(store, id) { // ลบ
  const db = await openDB();
  return new Promise((resolve, reject) => {
    const tx = db.transaction(store, "readwrite");
    tx.objectStore(store).delete(id);
    tx.oncomplete = () => resolve();
    tx.onerror = () => reject(tx.error);
  });
}
```

```
});  
}
```

รู้จัก async / await

การอ่านฐานข้อมูล "ใช้เวลา" JavaScript จึงไม่รอ แต่ให้ **Promise** (ใบสัญญาว่าจะส่งผลลัพธ์ให้ภายหลัง) เราใช้ **await** เพื่อ "รอกันว่าจะเสร็จ" ภายในฟังก์ชันที่ขึ้นต้นด้วย **async**

```
await dbPut("customers", { id: crypto.randomUUID(), name: "คุณสมหญิง" });  
const all = await dbGetAll("customers");  
console.log(all);
```

เกร็ดความรู้

ตอนนี้ให้ **ใช้งานได้ก่อน** ไม่ต้องเข้าใจทุกบรรทัด Promise เป็นหัวข้อที่ยากที่สุดของตอนนี้ ถ้าติดให้ถามผู้ปกครอง/ครู หรือทำตามตัวอย่างไปก่อน แล้วจะเข้าใจเองเมื่อได้ใช้บ่อย ๆ

ลองทำ (ทำตามทีละข้อ)

1. คัดลอกฟังก์ชันทั้ง 4 ไปไว้ใน db.js แล้วทดสอบใน Console ด้วยคำสั่งตัวอย่าง
2. เปิด F12 → Application → IndexedDB ฐานข้อมูล invoice-app และ store ที่สร้างไว้
3. ลองเปิดไฟล์ db.js ของแอปจริง สังเกตว่ามีฟังก์ชันคล้ายกันอยู่เป็นร้อย ๆ แต่หลักการเดียวกัน

ภารกิจท้าทาย

เขียนฟังก์ชัน `dbGet(store, id)` ที่อ่านข้อมูลขึ้นเดียวด้วย `objectStore(store).get(id)`

คำถามทบทวน

IndexedDB ดีกว่า localStorage อย่างไร (บอก 2 ข้อ)?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

13

ด้าน 4 · เก็บข้อมูลและสร้างหน้าหลัก

หน้าจัดการลูกค้า: เพิ่ม แก้ไข ลบ ค้นหา (CRUD)

เวลาที่แนะนำ: 90–120 นาที

เป้าหมาย: สร้างหน้าที่ครบวงจร CRUD คือ Create, Read, Update, Delete พร้อมช่องค้นหาดูไฟล์เหล่านี้ในแอปจริงประกอบ: `customers.html` `customer-import.html` `customer-sync.js`

ทุกหน้าในแอปธุรกิจคือ **CRUD** ในรูปแบบใดแบบหนึ่ง: สร้าง (Create) อ่าน (Read) แก้ไข (Update) ลบ (Delete) หน้า `customers.html` ของแอปจริงมีฟีเจอร์เพิ่มเช่น แท็บ ชิป กรอง และนำเข้าข้อมูล แต่แกนคือ 4 อย่างนี้

`customers.js`

```
let allCustomers = [];  
const searchBox = document.getElementById("search");  
const listEl = document.getElementById("customer-list");  
  
async function refresh() {  
  allCustomers = await dbGetAll("customers"); // READ  
  renderList();  
}  
  
function renderList() {  
  const q = searchBox.value.trim().toLowerCase();  
  const shown = allCustomers  
    .filter(c => c.name.toLowerCase().includes(q) || (c.phone || "").includes(q))  
    .sort((a, b) => a.name.localeCompare(b.name, "th"));  
  listEl.innerHTML = "";  
  for (const c of shown) {  
    const li = document.createElement("li");  
    li.textContent = `${c.name} ${c.phone || ""}`;  
    const edit = document.createElement("button");  
    edit.textContent = "แก้ไข";  
    edit.onclick = () => startEdit(c);  
    const del = document.createElement("button");  
    del.textContent = "ลบ";  
    del.onclick = () => removeCustomer(c);  
    li.append(edit, del);  
    listEl.appendChild(li);  
  }  
}  
  
async function saveCustomer(data) {  
  if (!data.id) data.id = crypto.randomUUID(); // CREATE  
  data.updatedAt = Date.now(); // UPDATE ใช้ id เดิม  
  await dbPut("customers", data);  
  await refresh();  
}  
  
async function removeCustomer(c) {  
  if (!confirm(`ลบ ${c.name} ใช่ไหม?`)) return; // DELETE  
  await dbDelete("customers", c.id);  
  await refresh();  
}  
  
searchBox.addEventListener("input", renderList);  
refresh();
```

เกร็ดความรู้

updatedAt คือเวลาที่ข้อมูลถูกแก้ไขล่าสุด เราใส่ไว้ตั้งแต่ตอนนี้ เพราะบทที่ 22 (Cloud Sync) ต้องใช้มัน

ระวัง!

ลูกค้าที่มีใบแจ้งหนี้อยู่ **ไม่ควรลบทิ้งจริง ๆ** เพราะประวัติบัญชีจะเสียหาย แอปจริงจึงมักใช้วิธี "ซ่อน/ยกเลิก" (ใส่ธง `cancelled: true` หรือ `deleted: true`) แทน

ลองทำ (ทำตามทีละข้อ)

1. ทำหน้า customers.html ให้ครบ: ฟอर्म + รายการ + ช่องค้นหา
2. ทดสอบเพิ่ม แก้ไข ลบ ค้นหา และรีเฟรชหน้าดูว่าข้อมูลอยู่ครบ
3. ค้นหาด้วยเบอร์โทรได้ไหม? ทดสอบ

ภารกิจท้าทาย

เปลี่ยนการลบให้เป็นแบบ **ซ่อน** (soft delete) โดยใส่ `deleted: true` และไม่แสดงลูกค้าที่ถูกซ่อนในรายการ

คำถามทบทวน

CRUD ย่อมาจากอะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

สินค้าและราคา: ราคามาตรฐาน vs ราคาเฉพาะลูกค้า

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: สร้างหน้าสินค้าและตรรกะหาราคาที่ถูกต้องสำหรับลูกค้าแต่ละคน

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `products.html` `pricing.html` `price-import.html` `pricing-sync.js` `pricing-grouped-sync.js`

ธุรกิจจริงมักมี ราคามาตรฐาน แต่ลูกค้าบางรายได้ ราคาพิเศษ แอปจริงมีไฟล์ `products.html`, `pricing.html` และระบบ `sync` ราคา ซึ่งเป็นส่วนที่ซับซ้อนที่สุดส่วนหนึ่ง เพราะต้องไม่ให้ราคาพังเมื่อแก้ไข

`pricing.js`

```
// สินค้า
const product = { id: "p1", name: "น้ำดื่ม", standardPrice: 12 };

// ลูกค้าที่มีราคาพิเศษ (เก็บเฉพาะสินค้าที่ราคาต่างจากมาตรฐาน)
const customer = { id: "c1", name: "ร้านคุณแดง", prices: { p1: 10 } };

function getPrice(product, customer) {
  const special = customer?.prices?.[product.id]; // ?. = ถ้าไม่มี ให้ได้ undefined ไม่ error
  if (special !== null) return special;           // มีราคาพิเศษ ใช้เลย
  return product.standardPrice;                  // ไม่มี ใช้ราคามาตรฐาน
}

console.log(getPrice(product, customer)); // 10
console.log(getPrice(product, null));     // 12
```

สถานการณ์	ราคาที่ใช้
ลูกค้ามีราคาพิเศษของสินค้านั้น	ราคาพิเศษ
ลูกค้าไม่มีราคาพิเศษ	ราคามาตรฐาน
ไม่ได้เลือกลูกค้า (ขายหน้าร้าน)	ราคามาตรฐาน

ระวัง!

อย่าแก้รายชื่อย้อนหลัง! ใบแจ้งหนี้ที่ออกไปแล้วต้อง คัดลอกราคา ณ วันนั้น ไว้ในใบแจ้งหนี้เอง ไม่อ้างอิงราคาปัจจุบัน ไม่เช่นนั้นเมื่อปรับราคาสินค้า ใบแจ้งหนี้เก่าจะเปลี่ยนตามและบัญชีจะผิด

ลองทำ (ทำตามทีละข้อ)

1. ทำหน้า `products.html` (CRUD เหมือนลูกค้า) เก็บใน store ใหม่ชื่อ `products`
2. เพิ่มช่อง "ราคาพิเศษ" ในหน้าลูกค้า
3. ทดสอบ `getPrice` ทั้ง 3 สถานการณ์ในตาราง

ภารกิจท้าทาย

เพิ่ม ราคาตามกลุ่มลูกค้า (เช่น ราคาขายส่ง/ปลีก) โดยลำดับความสำคัญคือ ราคาเฉพาะลูกค้า > ราคากลุ่ม > ราคามาตรฐาน

คำถามทบทวน

ทำไมใบแจ้งหนี้ต้องเก็บราคา ณ วันที่ออกไว้ในตัวเอง?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

สร้างใบแจ้งหนี้: เลขที่เอกสารและกันบันทึกซ้ำ

เวลาที่แนะนำ: 120 นาที

เป้าหมาย: ประกอบหน้าสร้างใบแจ้งหนี้ ออกเลขที่ไม่ซ้ำ และป้องกันการกดบันทึกซ้ำสองครั้งดูไฟล์เหล่านี้ในแอปจริงประกอบ: `invoice-create.html` `test-invoice-number-collision.js`
`test-double-save-guard.js`หน้า `invoice-create.html` เป็นหน้าที่ใหญ่ที่สุดของแอป (กว่า 100 KB) เพราะรวมทุกอย่างที่เรียนมา: เลือกลูกค้า เลือกสินค้า คำนวณราคา VAT ส่วนลด และบันทึก เราจะโฟกัส 2 เรื่องที่ทำให้ระบบ "น่าเชื่อถือ"

1) เลขที่ใบแจ้งหนี้ต้องไม่ซ้ำ

`invoice-number.js`

```

async function nextInvoiceNumber(year) {
  const all = await dbGetAll("invoices");
  const prefix = `INV${year}-`;
  const nums = all
    .filter(i => i.number && i.number.startsWith(prefix))
    .map(i => parseInt(i.number.slice(prefix.length), 10));
  const max = nums.length ? Math.max(...nums) : 0;
  return prefix + String(max + 1).padStart(4, "0"); // INV2026-0001
}

```

2) กันกดบันทึกซ้ำ (double-save guard)

ผู้ใช้ใจร้อนมักกดปุ่มบันทึก 2 ครั้ง ถ้าไม่กัน จะได้ใบแจ้งหนี้ 2 ใบ!

`save-invoice.js`

```

let saving = false;
async function saveInvoice(inv) {
  if (saving) return; // กำลังบันทึกอยู่ ไม่ทำซ้ำ
  saving = true;
  saveBtn.disabled = true;
  try {
    inv.id = inv.id || crypto.randomUUID();
    inv.number = inv.number || await nextInvoiceNumber(new Date().getFullYear());
    inv.updatedAt = Date.now();
    await dbPut("invoices", inv);
    location.href = "invoices.html";
  } finally {
    // ทำเสมอ แม้เกิดข้อผิดพลาด
    saving = false;
    saveBtn.disabled = false;
  }
}

```

ใบแจ้งหนี้ควรเก็บอะไรบ้าง	ตัวอย่าง
ข้อมูลหัวเอกสาร	<code>id, number, date, customerId, customerName</code>
รายการสินค้า (คัดลอกชื่อและราคา ณ วันนั้น)	<code>items: [{name, qty, price}]</code>
ยอดเงิน เก็บเป็นสตางค์ (อธิบายในบทที่ 17)	<code>subtotalSatang, vatSatang, totalSatang</code>

ใบแจ้งหนี้ควรเก็บอะไรบ้าง	ตัวอย่าง
สถานะและเวลา	cancelled, updatedAt

เกร็ดความรู้

ถ้ามีหลายเครื่องออกใบแจ้งหนี้พร้อมกัน เลขที่อาจชนกันได้! แอปจริงมีเทสต์ชื่อ `test-invoice-number-collision.js` เพื่อตรวจกรณีนี้ เราจะกลับมาเรื่องนี้ในบท Cloud Sync

ลองทำ (ทำตามทีละข้อ)

1. ประกอบหน้า invoice-create.html: เลือกลูกค้า → เพิ่มสินค้า → คำนวณ → บันทึก
2. ทดสอบกดบันทึกเร็ว ๆ ดูว่าได้ใบเดียว
3. สร้าง 3 ใบ ดูเลข INV2026-0001, 0002, 0003 ถูกต้องไหม

ภารกิจท้าทาย

ทำให้เลขที่รีเซตเป็น 0001 ทุกปีใหม่ (ใช้ prefix ที่มีปียอยู่แล้ว) และทดสอบโดยแก้ปีเป็นปีหน้า

คำถามทบทวน

`padStart(4, "0")` ทำอะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

รายการใบแจ้งหนี้ ตัวกรอง และการพิมพ์

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: แสดงรายการใบแจ้งหนี้ กรองตามสถานะ และพิมพ์เป็นเอกสารสวย ๆ

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `invoices.html` `history.html` `snapshots.html`

หน้า `invoices.html` แสดงทุกใบแจ้งหนี้ ผู้ใช้ต้องกรองได้ตามช่วงวันที่ ลูกค้า หรือสถานะ

`invoices.js`

```
function filterInvoices(all, { q = "", status = "all", from = "", to = "" }) {
  return all.filter(inv => {
    if (inv.cancelled) return false;
    if (status !== "all" && inv.status !== status) return false;
    if (from && inv.date < from) return false; // วันที่แบบ "2026-10-07" เทียบเป็นข้อความได้
    if (to && inv.date > to) return false;
    const text = `${inv.number} ${inv.customerName}`.toLowerCase();
    return text.includes(q.toLowerCase());
  });
}

const sorted = filterInvoices(all, opts).sort((a, b) => b.date.localeCompare(a.date));
```

พิมพ์ใบแจ้งหนี้ด้วย CSS

ไม่ต้องใช้โปรแกรมพิเศษ แค่เรียก `window.print()` แล้วใช้ CSS ซ่อนส่วนที่ไม่อยากพิมพ์

`style.css`

```
@media print {
  .nav, button, .no-print { display: none !important; }
  body { background: white; margin: 0; }
  .card { box-shadow: none; }
  @page { size: A4; margin: 15mm; }
}
```

```
<button onclick="window.print()" class="no-print">พิมพ์ / บันทึกเป็น PDF</button>
```

เกร็ดความรู้

ในหน้าต่างพิมพ์ของ Chrome เลือกปลายทางเป็น **Save as PDF** ก็ได้ไฟล์ PDF ฟรี แอปจริงมีหน้า `history.html` และ `snapshots.html` สำหรับดูประวัติ ซึ่งใช้หลักการกรอง/เรียงแบบเดียวกัน

ลองทำ (ทำตามทีละข้อ)

1. ทำหน้ารายการใบแจ้งหนี้พร้อมช่องค้นหา ตัวกรองสถานะ และวันที่
2. ลองพิมพ์ใบแจ้งหนี้ 1 ใบเป็น PDF
3. ใส่ชื่อร้าน ที่อยู่ ลายเซ็น ในหัว/ท้ายเอกสารให้ดูเป็นทางการ

ภารกิจท้าทาย

ทำให้คลิกหัวคอลัมน์ในตารางแล้วเรียงตามคอลัมน์นั้นได้ (กดซ้ำให้กลับด้าน)

คำถามทบทวน

@media print ใช้ทำอะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ด้าน 5

เงิน รายงาน และไฟล์

เป้าหมาย: จัดการเงินให้ถูกทุกสตางค์ ทำรายงาน และนำเข้า/ส่งออกไฟล์

สิ่งที่จะสร้าง: ระบบชำระเงิน รายงาน และ CSV

บทในด้านนี้

- บทที่ 17: ระบบชำระเงิน: ยอดค้าง เศษสตางค์ และสถานะ
- บทที่ 18: แดชบอร์ดและรายงาน: เปลี่ยนข้อมูลเป็นภาพ
- บทที่ 19: นำเข้าและส่งออกไฟล์: CSV และ Excel

ระบบชำระเงิน: ยอดค้าง เศษสตางค์ และสถานะ

เวลาที่แนะนำ: 120 นาที

เป้าหมาย: บันทึกการรับชำระ คำนวณยอดค้าง และเข้าใจว่าทำไมต้องเก็บเงินเป็นสตางค์

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `payments.html` `returns.html` `test-payment-rounding.js`
`test-outstanding-statement.js`

หน้า `payments.html` เป็นหน้าที่ใหญ่ที่สุดของแอป (ประมาณ 180 KB) เพราะเรื่องเงินต้อง ถูกต้องทุกสตางค์ มีทั้งเงินสด โอน เช็ค และการชำระบางส่วน

ภัยเงียบ: ทศนิยมใน JavaScript

ลองใน Console

```
console.log(0.1 + 0.2);           // 0.30000000000000004 (ไม่ใช่ 0.3!)
console.log(19.99 * 100);        // 1998.9999999999998
```

คอมพิวเตอร์เก็บทศนิยมแบบประมาณ จึงคลาดเคลื่อนเล็กน้อย ในงานเงินห้ามปล่อยไว้ วิธีแก้: เก็บเงินเป็นจำนวนเต็มของสตางค์ (100 สตางค์ = 1 บาท)

`payments.js`

```
const toSatang = baht => Math.round(baht * 100);
const toBaht   = satang => satang / 100;

// ใบแจ้งหนี้: totalSatang | การชำระ: { invoiceId, amountSatang, cancelled }
function paidAmount(inv, payments) {
  return payments
    .filter(p => p.invoiceId === inv.id && !p.cancelled)
    .reduce((sum, p) => sum + p.amountSatang, 0);
}

function balance(inv, payments) {
  return inv.totalSatang - paidAmount(inv, payments);
}

function statusOf(inv, payments) {
  if (inv.cancelled) return "cancelled";
  const b = balance(inv, payments);
  if (b <= 0) return "paid";           // จ่ายครบ
  if (b < inv.totalSatang) return "partial"; // จ่ายบางส่วน
  return "unpaid";                   // ยังไม่จ่าย
}
```

สถานะ	เงื่อนไข
unpaid (ค้างชำระ)	ยังไม่มีชำระเลย
partial (ชำระบางส่วน)	ชำระแล้วแต่ยังไม่ครบ
paid (ชำระครบ)	ยอดค้าง ≤ 0
cancelled (ยกเลิก)	ไม่นับรวมในยอดค้างและรายงาน

เกร็ดความรู้

แอปจริงมีเทสต์ชื่อ `test-payment-rounding.js`, `test-cancelled-excluded-from-unpaid.js` และ `test-outstanding-statement.js` ซึ่งเกิดจากบั๊กจริงที่เคยเจอ บั๊กเรื่องเงินคือสิ่งที่ทำให้ธุรกิจเสียหายได้ จึงต้องมีเทสต์ทุกจุด

ลองทำ (ทำตามทีละข้อ)

1. เพิ่ม store ชื่อ payments และทำฟอร์ม "รับชำระ" ในหน้าใบแจ้งหนี้
2. ทดสอบ: ใบ 1,000 บาท รับ 400 → สถานะ partial ยอดค้าง 600 / รับอีก 600 → paid
3. ยกเลิกการชำระหนึ่งรายการ ดูว่ายอดค้างกลับมาไหม

ภารกิจท้าทาย

ทำหน้า "ใบแจ้งหนี้ค้างชำระ" ที่รวมยอดค้างแยกตามลูกค้า เรียงจากมากไปน้อย

คำถามทบทวน

ทำไมจึงเก็บเงินเป็นสตางค์แทนบาทที่มีทศนิยม?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

แดชบอร์ดและรายงาน: เปลี่ยนข้อมูลเป็นภาพ

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: สรุปยอดขายรายเดือนและวาดกราฟแท่งง่าย ๆ ได้ด้วย HTML/CSS

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `dashboard.html` `reports.html`

เจ้าของร้านอยากรู้ว่า "เดือนนี้ขายได้เท่าไร ใครค้างเงินเยอะสุด" หน้า `dashboard.html` และ `reports.html` ตอบคำถามนี้ ด้วยเทคนิค `group` (จัดกลุ่ม) แล้วรวมยอด

`reports.js`

```
function totalsByMonth(invoices) {
  const out = {};
  for (const inv of invoices) {
    if (inv.cancelled) continue;
    const key = inv.date.slice(0, 7); // "2026-10"
    out[key] = (out[key] || 0) + inv.totalSatang;
  }
  return out; // { "2026-09": 125000, "2026-10": 98000 }
}

function drawBars(container, totals) {
  const entries = Object.entries(totals).sort();
  const max = Math.max(...entries.map([, v] => v), 1);
  container.innerHTML = "";
  for (const [month, value] of entries) {
    const row = document.createElement("div");
    row.className = "bar-row";
    row.innerHTML = `<span class="bar-label"></span><div class="bar"></div><span class="bar-val">
</span>`;
    row.querySelector(".bar-label").textContent = month;
    row.querySelector(".bar").style.width = (value / max * 100) + "%";
    row.querySelector(".bar-val").textContent = formatMoney(value / 100);
    container.appendChild(row);
  }
}
```

`style.css`

```
.bar-row { display:flex; align-items:center; gap:8px; margin:6px 0; }
.bar-label { width:70px; font-size:14px; }
.bar { height:20px; background:var(--main); border-radius:4px; transition:width .3s; }
```

เกร็ดความรู้

ถ้าอยากได้กราฟสวยกว่านี้ ใช้ไลบรารี เช่น `Chart.js` ได้ แต่การวาดแท่งด้วยตัวเองสอนให้เข้าใจว่ากราฟก็แค่ "ตัวเลขที่แปลงเป็นความกว้าง"

ลองทำ (ทำตามทีละข้อ)

1. ทำการ์ดสรุป 3 ใบ: ยอดขายเดือนนี้ ยอดค้างชำระ จำนวนลูกค้า
2. วาดกราฟแท่งยอดขายรายเดือนจากข้อมูลจริงที่ตัวเองสร้าง
3. เพิ่มตัวเลือกปี/เดือนเพื่อกรองรายงาน

ภารกิจท้าทาย

ทำรายการงาน "ลูกค้าซื้อเยอะสุด 5 อันดับ" (จัดกลุ่มตาม customerId, รวมยอด, เรียงลำดับ, ตัด 5 อันดับแรกด้วย slice(0,5))

คำถามทบทวน

ขั้นตอนทำรายการงาน 'ยอดรวมต่อกลุ่ม' คืออะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปพบต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

นำเข้าและส่งออกไฟล์: CSV และ Excel

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ส่งข้อมูลออกเป็นไฟล์ CSV เปิดใน Excel ได้ และนำเข้าข้อมูลจากไฟล์

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `customer-import.html` `excel-import.html` `price-import.html` `pdf-import.html`

ธุรกิจจริงมีข้อมูลอยู่ใน Excel อยู่แล้ว แอปจึงต้อง **นำเข้า (import)** ได้ และ **ส่งออก (export)** ให้นักบัญชีได้ แอปจริงมีหน้า `customer-import.html`, `excel-import.html`, `price-import.html` และ `pdf-import.html`

ส่งออก CSV

`export.js`

```
function downloadCSV(rows, filename) {
  const csv = rows
    .map(r => r.map(v => `${String(v).replace(/"/g, '""')}"`).join(","))
    .join("\n");
  const blob = new Blob(["\ufeff" + csv], { type: "text/csv;charset=utf-8" }); // \ufeff = BOM ให้ Excel
  อ่านไทยออก
  const a = document.createElement("a");
  a.href = URL.createObjectURL(blob);
  a.download = filename;
  a.click();
  URL.revokeObjectURL(a.href);
}

downloadCSV([["เลขที่", "ลูกค้า", "ยอด"], ["INV2026-0001", "คุณสมชาย", 1070]], "invoices.csv");
```

นำเข้า CSV

`import.js`

```
fileInput.addEventListener("change", async () => {
  const text = await fileInput.files[0].text();
  const lines = text.replace(/^\ufeff/, "").split(/\r?\n/).filter(Boolean);
  const [header, ...rows] = lines.map(l => l.split(","));
  console.log("หัวตาราง:", header);
  for (const r of rows) {
    await dbPut("customers", { id: crypto.randomUUID(), name: r[0].trim(), phone: (r[1] || "").trim() });
  }
});
```

ระวัง!

วิธี `split(",")` ง่าย ๆ จะพังถ้าข้อมูลมีเครื่องหมายจุลภาคอยู่ข้างใน เช่น "ห้างหุ้นส่วน, จำกัด" ในงานจริงให้ใช้ไลบรารีอ่าน CSV/Excel เช่น SheetJS และ **ตรวจสอบข้อมูลก่อนนำเข้าเสมอ** (แสดงตัวอย่างให้ผู้ใช้อยืนยันก่อนบันทึกจริง)

ลองทำ (ทำตามทีละข้อ)

1. ทำปุ่ม "ส่งออกลูกค้าเป็น CSV" แล้วเปิดใน Excel ดูภาษาไทย
2. ทำปุ่มนำเข้าลูกค้าจาก CSV ที่เตรียมไว้ 5 แถว
3. ลองเอาไฟล์ที่ส่งออกมา นำเข้ากลับเข้ามา ได้ข้อมูลซ้ำไหม?

ภารกิจท้าทาย

ก่อนบันทึกจริง ให้แสดงตารางตัวอย่าง 5 แถวแรกและปุ่ม "ยืนยันนำเข้า" และข้ามลูกค้าที่ซื้อซ้ำกับที่มีอยู่

คำถามทบทวน

ทำไมต้องใส่ \ufeff (BOM) หน้าไฟล์ CSV ภาษาไทย?

(เฉลยอยู่ในภาคผนวก จ)

เช็คลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ด้าน 6

แอประดับมืออาชีพ

เป้าหมาย: ทำให้แอปปลอดภัย ติดตั้งได้ ซิงก์ได้ ทดสอบได้ และดูแลต่อ
เนื่อง

สิ่งที่จะสร้าง: แอปครบวงจรระดับมืออาชีพ

บทในตอนนี้

- บทที่ 20: ล็อกอินและสิทธิ์ผู้ใช้
- บทที่ 21: PWA: ติดตั้งแอปบนมือถือและใช้ออฟไลน์
- บทที่ 22: Cloud Sync: ให้หลายเครื่องใช้ข้อมูลเดียวกัน
- บทที่ 23: สำรองและกู้คืนข้อมูล
- บทที่ 24: ทดสอบโปรแกรม: ไฟล์ test-*.js
- บทที่ 25: Git เก็บประวัติโค้ด และวิธีแก้บั๊ก
- บทที่ 26: ก้าวต่อไป: แอปเดสก์ท็อป (Tauri) และโครงสร้างโปรเจกต์ใหญ่

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: เข้าใจการเข้ารหัสรหัสผ่าน (hash) และกำหนดสิทธิ์ว่าใครทำอะไรได้บ้าง

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `auth.js` `users.html` `user-sync.js` `firestore.rules`

ถ้าแอปมีพนักงานหลายคน ต้องมีระบบ ล็อกอิน และ สิทธิ์ (role) เช่น เจ้าของดูรายงานได้ พนักงานขายออกใบแจ้งหนี้ได้แต่ลบไม่ได้ แอปจริงมี `auth.js`, `users.html` และ `user-sync.js`

ห้ามเก็บรหัสผ่านตรง ๆ

เราเก็บ hash (ลายนิ้วมือของรหัสผ่าน) ซึ่งแปลงกลับเป็นรหัสผ่านเดิมไม่ได้ เวลาล็อกอินก็ hash สิ่งที่พิมพ์มาแล้วเทียบกัน

`auth.js`

```
async function sha256(text) {
  const buf = await crypto.subtle.digest("SHA-256", new TextEncoder().encode(text));
  return [...new Uint8Array(buf)].map(b => b.toString(16).padStart(2, "0")).join("");
}

async function login(username, password, users) {
  const hash = await sha256(password);
  const user = users.find(u => u.username === username && u.passwordHash === hash);
  if (!user) return null;
  sessionStorage.setItem("currentUser", JSON.stringify({ id: user.id, role: user.role }));
  return user;
}

const PERMISSIONS = {
  owner: ["view_reports", "create_invoice", "delete_invoice", "manage_users"],
  sales: ["create_invoice"],
};

function can(user, action) {
  return !!user && (PERMISSIONS[user.role] || []).includes(action);
}
```

ระวัง!

ความจริงที่ต้องรู้: แอปที่ทำงานในเบราว์เซอร์อย่างเดียว ผู้ใช้เก่ง ๆ เปิด DevTools ข้ามการตรวจสอบสิทธิ์ได้ ดังนั้นการซ่อนปุ่มเป็นแค่ "ความสะดวก" ไม่ใช่ความปลอดภัยจริง ความปลอดภัยจริงต้องมี **กฎฝั่งเซิร์ฟเวอร์** (เช่น Firestore rules ในบทที่ 22) และห้ามใส่ความลับสำคัญไว้ในโค้ดหน้าเว็บ

เกร็ดความรู้

`crypto.subtle` ใช้ได้เฉพาะบนหน้าเว็บที่เปิดผ่าน `https` หรือ `localhost` (ซึ่ง Live Server ทำให้อยู่แล้ว) ระบบจริงควรใช้การ hash แบบมี salt และซ้ำ เช่น `bcrypt/PBKDF2` ฝั่งเซิร์ฟเวอร์

ลองทำ (ทำตามทีละข้อ)

1. ทำหน้า login ง่าย ๆ และสร้างผู้ใช้ owner / sales ในฐานข้อมูล
2. ให้ปุ่มลบใบแจ้งหนี้แสดงเฉพาะผู้ใช้ที่ `can(user, "delete_invoice")` เป็นจริง
3. ทดสอบล็อกอินด้วยรหัสผิด ต้องไม่ผ่าน

ภารกิจท้าทาย

ให้หน้าที่ต้องล็อกอินดึงกลับไปหน้า login อัตโนมัติถ้ายังไม่ได้ล็อกอิน (เช็คใน nav.js)

คำถามทบทวน

ทำไมไม่ควรเก็บรหัสผ่านตรง ๆ ในฐานข้อมูล?

(เฉลยอยู่ในภาคผนวก จ)

เช็คลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

PWA: ติดตั้งแอปบนมือถือและใช้ออฟไลน์

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ทำให้เว็บของเรากลายเป็นแอปที่ติดตั้งบนหน้าจอโฮมและทำงานได้โดยไม่มีอินเทอร์เน็ต

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `manifest.json` `sw.js` `icons/` `versions.html` `update-version.bat`

PWA (Progressive Web App) คือเว็บที่ทำตัวเหมือนแอป ใช้ได้ 2 ไฟล์หลัก: **manifest.json** (บัตรประจำตัวแอป) และ **sw.js** (Service Worker คือ "ผู้ช่วยหลังบ้าน" ที่เก็บไฟล์ไว้ใช้ตอนออฟไลน์)

manifest.json

```
{
  "name": "Invoice App",
  "short_name": "Invoices",
  "start_url": "./index.html",
  "display": "standalone",
  "background_color": "#f1f5f9",
  "theme_color": "#2563eb",
  "icons": [
    { "src": "icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}
```

ใส่ใน index.html

```
<link rel="manifest" href="manifest.json">
<meta name="theme-color" content="#2563eb">
<script>
  if ("serviceWorker" in navigator) navigator.serviceWorker.register("sw.js");
</script>
```

sw.js

```
const CACHE = "invoice-v1"; // เปลี่ยนเลขเวอร์ชันทุกครั้งที่อัปเดตไฟล์
const FILES = ["/", "./index.html", "./style.css", "./utils.js", "./nav.js", "./db.js"];

self.addEventListener("install", e => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(FILES)));
  self.skipWaiting();
});
self.addEventListener("activate", e => {
  e.waitUntil(
    caches.keys().then(keys => Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k))))
  );
  self.clients.claim();
});
self.addEventListener("fetch", e => {
  e.respondWith(caches.match(e.request).then(r => r || fetch(e.request)));
});
```

ระวัง!

ปัญหาอดฮิต: แก้โค้ดแล้ว แต่หน้าเว็บยังเป็นของเก่า! เพราะ Service Worker เก็บไฟล์เก่าไว้ ให้เปลี่ยนเลข CACHE (v1 → v2) หรือใน DevTools → Application → Service Workers ดึง Update on reload ตอนพัฒนา แอปจริง มีหน้า versions.html และ update-version.bat ไว้ช่วยจัดการเรื่องเวอร์ชัน

ลองทำ (ทำตามทีละข้อ)

1. เพิ่ม manifest.json และ sw.js ให้โปรเจกต์ แล้วสร้างไอคอน 192 และ 512 พิกเซล
2. เปิด DevTools → Application → Manifest ดูว่าไม่มีข้อผิดพลาด
3. ปิดเน็ต (หรือติ๊ก Offline ในแท็บ Network) แล้วรีเฟรช แอปต้องยังเปิดได้

ภารกิจท้าทาย

ทำให้แสดงข้อความ "ออฟไลน์อยู่" เมื่ออินเทอร์เน็ตหลุด (ใช้: เหตุการณ์ `window.addEventListener("offline", ...)`)

คำถามทบทวน

Service Worker ทำอะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

Cloud Sync: ให้หลายเครื่องใช้ข้อมูลเดียวกัน

เวลาที่แนะนำ: 120 นาที

เป้าหมาย: เข้าใจแนวทางการซิงค์ข้อมูลขึ้นคลาวด์ การรวมข้อมูล (merge) และการเก็บความลับให้ปลอดภัย

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `sync.js` `collection-sync.js` `customer-sync.js` `product-sync.js`
`drive-store.js` `firebase-config.example.js` `firestore.rules` `connection-status.js`

ข้อมูลที่เก็บใน IndexedDB อยู่ใน **เครื่องเดียว** ถ้าเจ้าของร้านอยากดูข้อมูลบนมือถือและคอมพิวเตอร์ ต้องมีที่กลางบนคลาวด์ แอปจริงรองรับหลายแบบ (ดูไฟล์ `sync.js`, `firebase-config.js`, `drive-store.js`, `collection-sync.js`) ซึ่งเป็นส่วนที่ใหญ่และซับซ้อนที่สุดของระบบ

เปรียบเทียบให้เห็นภาพ

เหมือนสมุดบัญชีเล่มกลางที่ทุกคนคัดลอกไปใช้ได้ ปัญหาคือถ้า **สองคนแก้บรรทัดเดียวกันพร้อมกัน** จะเชื่อใคร? ต้องมีกติกาตกลงกัน

กติกาง่าย ๆ: ฉบับล่าสุดชนะ (last-write-wins)

```
sync-idea.js

// ทุกชิ้นข้อมูลต้องมี id และ updatedAt (เราใส่ไว้ตั้งแต่บทที่ 13 แล้ว)
function merge(local, remote) {
  const map = new Map(local.map(x => [x.id, x]));
  for (const r of remote) {
    const l = map.get(r.id);
    if (!l || r.updatedAt > l.updatedAt) map.set(r.id, r); // ผิงใหม่ดีกว่าชนะ
  }
  return [...map.values()];
}

// ซิงค์เฉพาะที่เปลี่ยนตั้งแต่ครั้งล่าสุด ช่วยประหยัดโควตาและเงิน
const changed = local.filter(x => x.updatedAt > lastSyncTime);
```

ระวัง!

การลบต้องระวัง! ถ้าเครื่อง A ลบข้อมูล แล้วเครื่อง B ที่ยังมีข้อมูลนั้นซิงค์ ข้อมูลที่ลบอาจ "ฟื้นคืนชีพ" วิธีแก้คือไม่ลบจริง แต่ใส่ธง `deleted: true` และ `updatedAt` ใหม่ (เรียกว่า **tombstone**) แล้วซิงค์ธงนั้นไปด้วย

ตัวอย่างโครงสร้าง Firebase Firestore (ภาพรวม)

```
firebase-example.js

// ตรวจสอบเวอร์ชัน SDK ล่าสุดจากเอกสาร Firebase ก่อนใช้งานจริง
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
import { getFirestore, doc, setDoc } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";

const app = initializeApp(firebaseConfig); // firebaseConfig มาจากหน้าตั้งค่าโปรเจกต์
const cloud = getFirestore(app);

async function pushCustomer(c) {
```

```
await setDoc(doc(cloud, "customers", c.id), c);
}
```

firebase.rules (ตัวอย่างพื้นฐาน)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null; // ต้องล็อกอินก่อน
    }
  }
}
```

กฎหลักเรื่องความปลอดภัย

- ห้ามอัปโหลดไฟล์ที่มีรหัสลับ/credentials ขึ้นที่สาธารณะ (เช่น GitHub แบบ public) ให้ใส่ชื่อไฟล์เหล่านั้นใน .gitignore เสมอ แอปจริงมีไฟล์ตัวอย่างเช่น firebase-config.example.js ไว้ให้ดูโครงสร้างโดยไม่มีค่าจริง
- ทุกบริการคลาวด์มี โควตา (จำนวนอ่าน/เขียนต่อวัน) การซิงก์ทุกอย่างทุกครั้งเปลืองมาก ต้องซิงก์เฉพาะส่วนที่เปลี่ยน
- ความปลอดภัยจริงอยู่ที่ กฎฝั่งเซิร์ฟเวอร์ (rules) ไม่ใช่การซ่อนไว้ในโค้ดหน้าเว็บ

ลองทำ (ทำตามทีละข้อ)

- เขียนฟังก์ชัน merge และทดสอบด้วยข้อมูลจำลอง 2 ชุดที่แก๊สคู่คณเดียวกันคนละเวลา
- สร้างโปรเจกต์ Firebase ฟรีของตัวเอง (ให้ผู้ปกครองช่วยดูแลบัญชี) และลองเขียนข้อมูล 1 รายการขึ้น Firestore
- เปิดไฟล์ .gitignore ของแอปจริง ดูว่าไฟล์ประเภทไหนถูกกันไว้

ภารกิจท้าทาย

ทำปุ่ม "ซิงก์ตอนนี้" ที่แสดงจำนวนรายการที่ส่งขึ้น/ติงลง และเวลาซิงก์ล่าสุด (เก็บ lastSyncTime ไว้ใน localStorage)

คำถามทบทวน

ทำไมต้องมี updatedAt ในข้อมูลทุกชิ้น?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: สร้างระบบสำรองข้อมูล (backup) และกันความผิดพลาดจากการลบที่เดียวหลายรายการ

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `snapshots.html` `test-rotating-restore-point.js` `test-mass-delete-guard.js` `test-explicit-delete-logging.js` `local-folder-sync.js`

ข้อมูลธุรกิจคือสมบัติ ระบบที่ดีต้องตอบได้ว่า "ถ้าลบผิด หรือเครื่องพัง จะกู้ได้ไหม" แอปจริงมีหน้า `snapshots.html` และเทสต์เกี่ยวกับ restore point (`test-rotating-restore-point.js`) และ mass-delete guard (`test-mass-delete-guard.js`)

`backup.js`

```
const STORES = ["customers", "products", "invoices", "payments"];

async function exportAll() {
  const data = { version: 1, exportedAt: new Date().toISOString() };
  for (const s of STORES) data[s] = await dbGetAll(s);
  const blob = new Blob([JSON.stringify(data, null, 2)], { type: "application/json" });
  const a = document.createElement("a");
  a.href = URL.createObjectURL(blob);
  a.download = `backup-${data.exportedAt.slice(0, 10)}.json`;
  a.click();
  URL.revokeObjectURL(a.href);
}

async function importAll(file) {
  const data = JSON.parse(await file.text());
  if (data.version !== 1) throw new Error("ไฟล์สำรองไม่ถูกต้อง");
  for (const s of STORES) for (const item of data[s] || []) await dbPut(s, item);
}
```

3 เกราะป้องกัน

เกราะ	แนวคิด
Restore point หมุนเวียน	เก็บสำเนาสำรองล่าสุดไว้ N ชุด (เช่น 5 ชุด) เมื่อเต็มก็ลบชุดเก่าสุดทิ้ง ไม่ให้พื้นที่บวม
Mass-delete guard	ถ้าผู้ใช้จะลบ > 10 รายการพร้อมกัน ให้ถามยืนยัน (เช่น ให้พิมพ์คำว่า DELETE) และสำรองก่อนลบ
บันทึกการลบ (log)	ทุกการลบต้องมีบันทึกว่าใคร ลบอะไร เมื่อไร (ดู <code>test-explicit-delete-logging.js</code>)

`guard.js`

```
function confirmMassDelete(count) {
  if (count <= 10) return true;
  return prompt(`จะลบ ${count} รายการ! พิมพ์ DELETE เพื่อยืนยัน`) === "DELETE";
}
```

ลองทำ (ทำตามทีละข้อ)

1. ทำปุ่มสำรองและกู้คืน ทดสอบโดยสำรอง → ลบข้อมูลทั้งหมด → กู้คืน
2. ลองนำเข้าไฟล์ที่ผิดรูปแบบ แอปต้องแจ้งเตือนและไม่พัง
3. ตั้งเป้า: สำรองข้อมูลทุกสัปดาห์ เก็บไว้อย่างน้อย 2 ที่ (คอม + คลาวด์/แฟลชไดรฟ์)

ภารกิจท้าทาย

ทำระบบสำรองอัตโนมัติ: เมื่อเปิดแอป ถ้าครั้งล่าสุดผ่านไปเกิน 7 วัน ให้เตือนให้สำรอง

คำถามทบทวน

กฎ 3-2-1 ของการสำรองข้อมูลคืออะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ทดสอบโปรแกรม: ไฟล์ test-*.js

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: เขียนเทสต์อัตโนมัติแรกของตัวเอง และเข้าใจว่าทำไมแอปจริงมีไฟล์เทสต์เป็นสิบ ๆ ไฟล์

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `test-payment-rounding.js` `test-invoice-number-collision.js`
`test-double-save-guard.js` `test-stale-balance-flag.js` `test-pricing-roundtrip.js`

ลองสังเกตโฟลเดอร์ของแอปจริง จะเห็นไฟล์ชื่อ test-*.js เป็นสิบ ๆ ไฟล์ เช่น test-payment-rounding.js, test-invoice-number-collision.js แต่ละไฟล์มักเกิดจาก **บั๊กจริง** ที่เคยเจอ นักพัฒนาเขียนเทสต์ไว้ เพื่อให้บั๊กเดิมไม่กลับมาอีก

เปรียบเทียบให้เห็นภาพ

เทสต์ = ยามเฝ้าประตู ทุกครั้งที่เราแก้โค้ด ยามจะตรวจทุกเรื่องที่เคยพังมาแล้วให้ ถ้าเราไปทำของเดิมพัง ยามจะร้องทันที

```
test-money.js

// test-money.js วิธีรัน: node test-money.js
const assert = require("assert");

const toSatang = baht => Math.round(baht * 100);

function statusOf(totalSatang, paidSatang, cancelled) {
  if (cancelled) return "cancelled";
  const b = totalSatang - paidSatang;
  if (b <= 0) return "paid";
  if (b < totalSatang) return "partial";
  return "unpaid";
}

assert.strictEqual(toSatang(0.1 + 0.2), 30); // ปิดเศษลูก
assert.strictEqual(toSatang(19.99), 1999);
assert.strictEqual(statusOf(100000, 0, false), "unpaid");
assert.strictEqual(statusOf(100000, 40000, false), "partial");
assert.strictEqual(statusOf(100000, 100000, false), "paid");
assert.strictEqual(statusOf(100000, 0, true), "cancelled");

console.log("ผ่านทุกข้อ!");
```

ขั้นตอน "เจอบั๊ก แล้วทำให้มันไม่กลับมา"

- ทำซ้ำได้: หาวิธีที่ทำให้บั๊กเกิดทุกครั้ง
- เขียนเทสต์ที่ล้มเหลว: เขียนเทสต์ที่จับบั๊กนี้ให้ได้ (ตอนแรกเทสต์ต้อง "แดง")
- แก้โค้ด จนเทสต์เป็น "เขียว"
- รันเทสต์ทั้งหมด ให้แน่ใจว่าไม่ได้ทำของเก่าพัง

เกร็ดความรู้

ลองคัดลอกฟังก์ชัน toSatang จากโค้ดจริงมาแปะในไฟล์ทดสอบ แล้วลองทดสอบค่าแปลก ๆ เช่น 0, ตัวเลขติดลบ, 1e-7 บางทีเราจะเจอกรณีที่ไม่เคยคิดถึง!

ลองทำ (ทำตามทีละข้อ)

1. ติดตั้ง Node.js แล้วรัน `node test-money.js` ให้เห็นข้อความ "ผ่านทุกข้อ!"
2. แก้งแก้โค้ดให้ผิด (เช่น เปลี่ยน `<=` เป็น `<`) แล้วรันใหม่ ดูว่าทดสอบจับได้ไหม
3. เปิดไฟล์ `test-*.js` ของแอปจริง 1 ไฟล์ อ่านแค่ชื่อฟังก์ชันและคอมเมนต์ให้เข้าใจว่าทดสอบเรื่องอะไร

ภารกิจท้าทาย

เขียนเทสต์ 5 ข้อสำหรับฟังก์ชัน `nextInvoiceNumber` (ข้อมูลว่าง, มี 1 ใบ, มีเลขข้ามลำดับ, ข้ามปี, มีเลขผิดรูปแบบปนอยู่)

คำถามทบทวน

ทำไมควรเขียนเทสต์หลังเจอบั๊ก?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

เวลาที่แนะนำ: 60–90 นาที

เป้าหมาย: ใช้ Git บันทึกเวอร์ชันของงาน และมีขั้นตอนแก้ปัญหาย่างเป็นระบบด้วย DevTools

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `.git/` `.gitignore` `troubleshoot.html` `help.html` `versions.html`

Git คือเครื่องย้อนเวลาของโค้ด ทุกครั้งที่ "commit" คือการถ่ายภาพโปรเจกต์ไว้ ถ้าแก้พัง ย้อนกลับได้เสมอ (แอปจริงมีโฟลเดอร์ `.git` อยู่แล้ว)

Terminal

```
git init                # เริ่มเก็บประวัติในโฟลเดอร์นี้ (ทำครั้งเดียว)
git status              # ดูว่ามีอะไรเปลี่ยน
git add .               # เลือกไฟล์ที่เปลี่ยนทั้งหมดไว้ถ่ายภาพ
git commit -m "เพิ่มหน้าลูกค้า" # ถ่ายภาพ พร้อมข้อความอธิบาย
git log --oneline       # ดูประวัติ

git switch -c idea-pricing # แดกกิ่งไปลองไอเดียใหม่โดยไม่กระทบของเดิม
git switch main          # กลับกิ่งหลัก
```

เครื่องมือช่วยวินิจฉัย

เมื่อโปรแกรมมีปัญหา ให้ใช้ ขั้นตอนวินิจฉัยใน ภาคผนวก ง ซึ่งมีผังตัดสินใจและสคริปต์ตรวจสอบสภาพแอปให้วางใน Console ได้เลย

- commit บ่อย ๆ ครั้งละเรื่องเดียว และเขียนข้อความให้เข้าใจ
- ก่อนทำอะไรเสี่ยง ให้ commit ไว้ก่อน
- ห้าม commit ไฟล์ความลับ (ตรวจ `.gitignore`)

5 ขั้นตอนแก้บั๊ก (ใช้ได้ตลอดชีวิตนักเขียนโปรแกรม)

ขั้น	ทำอะไร	เครื่องมือ
1. อ่านข้อความ error	คำตอบมักอยู่ในข้อความสีแดง บอกชื่อไฟล์และบรรทัด	แท็บ Console
2. ทำซ้ำให้ได้	หาขั้นตอนที่ทำให้เกิดบั๊กทุกครั้ง	จดขั้นตอน
3. ตรวจสอบค่า	พิมพ์ค่าตัวแปรดูว่าตรงที่คิดไหม	console.log, debugger
4. ตรวจสอบมูลจริง	ดูข้อมูลที่เก็บอยู่ ไฟล์ที่โหลด คำขอเครือข่าย	Application, Network
5. แก้ทีละอย่าง	แก้อย่างเดียว แล้วลองใหม่ ถ้าไม่หายค่อยย้อนกลับ	Git

เกร็ดความรู้

แอปจริงมีหน้า `troubleshoot.html` และ `help.html` สำหรับผู้ใช้ปลายทาง ลองเปิดอ่านเพื่อดูว่า "เอกสารช่วยเหลือที่ดี" เขียนอย่างไร การเขียนวิธีแก้ปัญหาก็เป็นทักษะสำคัญของนักพัฒนา

ลองทำ (ทำตามทีละข้อ)

1. ติดตั้ง Git แล้วรัน `git init` ในโฟลเดอร์โปรเจกต์ของเรา และ commit ครั้งแรก
2. แก้งทำให้ `app.js` พัง แล้วใช้ `git diff` ดูว่าเปลี่ยนอะไร จากนั้นย้อนด้วย `git restore app.js`
3. จงแก้บั๊กที่ครู/ผู้ปกครองแอบใส่ไว้ในโค้ด 1 จุด โดยใช้ 5 ขั้นตอนข้างบน

ภารกิจท้าทาย

สร้างบัญชี GitHub (ให้ผู้ปกครองช่วย) แล้วอัปโหลดโปรเจกต์เป็น **private repository**

คำถามทบทวน

commit คืออะไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กलिस्टก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ก้าวต่อไป: แอปเดสก์ท็อป (Tauri) และโครงสร้างโปรเจกต์ใหญ่

เวลาที่แนะนำ: 60 นาที

เป้าหมาย: รู้ภาพรวมว่าแอปเว็บแปลงเป็นโปรแกรมบนคอมพิวเตอร์ได้อย่างไร และอ่านโครงสร้างโปรเจกต์ใหญ่ได้

ดูไฟล์เหล่านี้ในแอปจริงประกอบ: `src-tauri/` `returns.html` `settings.html`

ในโพลเดอร์แอปจริงมี `src-tauri` ซึ่งคือ Tauri เครื่องมือห่อเว็บแอปของเราให้กลายเป็นโปรแกรม .exe/.app ที่ติดตั้งบนคอมพิวเตอร์ ใช้ HTML/CSS/JS เหมือนเดิมทุกอย่าง แต่ส่วนห่อหุ้มเขียนด้วยภาษา Rust

ชนิดแอป	เหมาะกับ	ในโปรเจกต์นี้
เว็บ/PWA	เปิดผ่านเบราว์เซอร์ ติดตั้งจากหน้าเว็บ ใช้ได้ทุกเครื่อง	ไฟล์ .html + manifest.json + sw.js
เดสก์ท็อป (Tauri)	ต้องการเข้าถึงไฟล์ในเครื่อง ทำงานเหมือนโปรแกรมทั่วไป	src-tauri/

อ่านโครงสร้างโปรเจกต์ใหญ่ให้เป็น

เมื่อเจอโค้ดใหญ่ ๆ อย่าตกใจ ให้ถาม 3 ข้อ: (1) จุดเข้าอยู่ไหน (index.html) (2) หน้าไหนทำหน้าที่อะไร (ดูชื่อไฟล์) (3) ไฟล์ไหน "ใช้ร่วมกัน" (nav.js, utils.js, db.js, style.css) ลองเขียนแผนที่ของตัวเองลงในภาคผนวก ก

จุดหมายปลายทางของคอร์ส

ใช้ความรู้ที่เรียนมา อ่านโค้ดของแอปจริง 1 หน้า (เริ่มจาก customers.html หรือ products.html ซึ่งเล็กกว่า) แล้วอธิบายได้ว่าแต่ละส่วนทำอะไร ไม่จำเป็นต้องเข้าใจทุกบรรทัด

ลองทำ (ทำตามทีละข้อ)

1. เปิดแอปจริง 1 หน้า ใช้ Ctrl+F หาคำว่า `addEventListener` กับ `dbPut` ดูว่าเกิดขึ้นตรงไหน
2. วาดแผนภาพ (บนกระดาษ) ว่าข้อมูลไหลจากฟอร์ม → db → ตาราง อย่างไร
3. เลือกฟีเจอร์ 1 อย่างในแอปจริง แล้วลองสร้างเวอร์ชันเล็กของตัวเองใน my-invoice-app

ภารกิจท้าทาย

ฟีเจอร์ท้าทาย: ทำ **ใบลดหนี้/คืนสินค้า** (ดูไอเดียจาก returns.html) ที่หักยอดจากใบแจ้งหนี้เดิม

คำถามทบทวน

PWA กับแอป Tauri ต่างกันอย่างไร?

(เฉลยอยู่ในภาคผนวก จ)

เช็กลิสต์ก่อนไปบทต่อไป

- ☐ ฉันทำ "ลองทำ" ครบทุกข้อและโปรแกรมทำงานได้
- ☐ ฉันอธิบายสิ่งที่เรียนในบทนี้ให้คนอื่นฟังได้
- ☐ ฉันพยายามทำภารกิจท้าทายแล้ว (สำเร็จหรือไม่ก็ได้)

ภาคผนวก ก: แผนที่ไฟล์ของแอปจริง

ตารางนี้จับคู่ไฟล์ในโปรเจกต์ตัวเต็มกับบทเรียน ใช้เป็นแผนที่ตอนไปสำรวจโค้ดจริง (หมายเหตุสำหรับผู้สอน: จัดทำจากรายชื่อและขนาดไฟล์ในโฟลเดอร์ หน้าที่ของแต่ละไฟล์เป็นการอนุมานจากชื่อ ควรเปิดตรวจกับโค้ดจริงก่อนสอนเสมอ)

ไฟล์	หน้าที่โดยสังเขป	บทที่เกี่ยวข้อง
index.html	หน้าแรก / จุดเริ่มต้นแอป	2, 21
style.css	หน้าตาที่ใช้ร่วมกันทั้งแอป	4, 5, 16
nav.js	สร้างเมนูนำทางให้ทุกหน้า	5, 10
utils.js	ฟังก์ชันช่วย (จัดรูปแบบเงิน/วันที่ ฯลฯ)	6, 7, 10
idb.js / db.js	ชั้นฐานข้อมูล IndexedDB	12
customers.html	จัดการลูกค้า	13
customer-import.html	นำเข้าลูกค้าจากไฟล์	19
products.html	จัดการสินค้า	14
pricing.html / price-import.html	ราคาและนำเข้าราคา	14, 19
invoice-create.html	สร้าง/แก้ไขใบแจ้งหนี้	9, 15
invoices.html	รายการใบแจ้งหนี้	16
payments.html	รับชำระเงิน ยอดค้าง	17
returns.html	คืนสินค้า/ลดหนี้	26
dashboard.html	แดชบอร์ดสรุป	5, 18
reports.html	รายงาน	18
excel-import.html / pdf-import.html	นำเข้าจาก Excel / PDF	19
auth.js / users.html	ล็อกอินและจัดการผู้ใช้	20
settings.html / settings.js	ตั้งค่าแอป	11, 26
history.html / snapshots.html	ประวัติและจุดกู้คืน	16, 23
manifest.json / sw.js / icons	ทำให้เป็น PWA ใช้ออฟไลน์	21
sync.js / collection-sync.js ฯลฯ	ซิงค์ข้อมูลขึ้นคลาวด์	22
firebase-config*.js / firestore.rules	ตั้งค่าและกฎความปลอดภัยของ Firebase	22
test-*.js	ชุดทดสอบอัตโนมัติ	17, 24
versions.html / update-version.bat	จัดการเวอร์ชันแอป	21, 25
help.html / troubleshoot.html	คู่มือผู้ใช้และแก้ปัญหา	2, 25
src-tauri/	ห่อแอปเป็นโปรแกรมเดสก์ท็อป	26

ไฟล์	หน้าที่โดยสังเขป	บทที่เกี่ยวข้อง
<code>.git / .gitignore</code>	ประวัติโค้ดและรายการไฟล์ที่ไม่เก็บ	22, 25

ภาคผนวก ข: อภิธานศัพท์ (ไทย-อังกฤษ)

คำศัพท์	ความหมาย
Browser	โปรแกรมเปิดเว็บ เช่น Chrome
HTML	ภาษาที่บอกว่าหน้าเว็บมีอะไรบ้าง
CSS	ภาษาที่ตกแต่งหน้าตา
JavaScript	ภาษาที่ทำให้เว็บทำงานและตอบสนอง
Tag / แท็ก	ป้ายกำกับใน HTML เช่น <p>
Variable / ตัวแปร	กล่องเก็บค่า มีชื่อกำกับ
Function / ฟังก์ชัน	ชุดคำสั่งที่เรียกใช้ซ้ำได้
Array	รายการของหลายสิ่ง
Object	ข้อมูลหนึ่งชิ้นที่มีหลายช่อง เช่น สินค้า
DOM	โครงสร้างองค์ประกอบของหน้าเว็บที่ JS แก้ไขได้
Event / เหตุการณ์	สิ่งที่เกิดขึ้น เช่น คลิก พิมพ์
CRUD	สร้าง อ่าน แก้ไข ลบ
localStorage	ที่เก็บข้อมูลเล็ก ๆ ในเบราว์เซอร์
IndexedDB	ฐานข้อมูลในเบราว์เซอร์
Promise / async / await	รูปแบบเขียนโค้ดที่ต้องรอผลลัพธ์
Responsive	หน้าเว็บปรับตามขนาดจอ
PWA	เว็บที่ติดตั้งและใช้ออฟไลน์ได้เหมือนแอป
Service Worker	ผู้ช่วยเบื้องหลังที่เก็บไฟล์ไว้ใช้ออฟไลน์
Hash	ค่าที่แปลงจากข้อมูล ย้อนกลับไม่ได้ ใช้เก็บรหัสผ่าน
Sync	ทำให้ข้อมูลหลายที่ตรงกัน
Merge	รวมข้อมูลสองชุดเป็นชุดเดียว
Tombstone	ธงที่ใช้แทนการลบจริง เพื่อให้ซิงก์การลบได้
Backup	สำเนาสำรองข้อมูล
Bug	ข้อผิดพลาดในโปรแกรม
Debug	การหาและแก้บั๊ก
Test / ทดสอบ	โค้ดที่ตรวจสอบว่าโปรแกรมทำงานถูกต้อง
Git / Commit	ระบบเก็บประวัติโค้ด / การบันทึกหนึ่งเวอร์ชัน
Repository (repo)	โพลเดอร์โปรเจกต์ที่ Git ดูแล

คำศัพท์	ความหมาย
XSS	การแอบแทรกโค้ดอันตรายผ่านช่องกรอกข้อมูล
VAT	ภาษีมูลค่าเพิ่ม (ไทย 7%)

ภาคผนวก ค: ปัญหาที่พบบ่อยและวิธีแก้

อาการ	วิธีแก้
หน้าเว็บขาว ไม่แสดงอะไร	เปิด F12 → Console อ่านข้อความสีแดง มักสะกดผิดหรือลืมปิดวงเล็บ
ภาษาไทยกลายเป็นตัวประหลาด	ลืม meta charset UTF-8 หรือบันทึกไฟล์ไม่เป็น UTF-8
แก้ CSS/JS แล้วไม่เปลี่ยน	แคชค้าง: กด Ctrl+Shift+R, หรือเปลี่ยนเวอร์ชัน cache ใน sw.js
Cannot read properties of null	JS หา id ไม่เจอ: id สะกดผิด หรือ script อยู่ก่อน HTML ให้ย้าย script ไปท้าย body
ข้อมูลหายหลังรีเฟรช	ยังไม่ได้บันทึกลง localStorage/IndexedDB หรือ await ไม่ครบ
x is not defined	ใช้ฟังก์ชัน/ตัวแปรก่อนประกาศ หรือลืมโหลดไฟล์ที่ประกาศไว้ (เช็กลำดับ script)
ปุ่มกดแล้วไม่ทำอะไร	ลืม addEventListener หรือผูกกับ id ผิด ลอง console.log ใน handler
ยอดเงินมีทศนิยมยาวเพี้ยน	ใช้ทศนิยมจำนวน ให้เปลี่ยนเป็นเก็บเป็นสตริง (บทที่ 17)
ใช้งานแล้ว git ไม่ยอม commit	ตรวจ git status; ต้อง git add ก่อน และตั้งชื่อ/อีเมลผู้ใช้ด้วย git config

เคล็ดลับของนักแก้บั๊ก

ก่อนถามใคร ให้ลองทำ 3 อย่างนี้: (1) อ่านข้อความ error ทั้งหมด (2) ค้นหาข้อความ error นั้นใน Google (3) ลองโค้ดให้เหลือส่วนที่พังอย่างเดียว บ่อยครั้งแค่ขั้นที่ 3 ก็เจอคำตอบแล้ว

ภาคผนวก ง: คู่มือผู้ปกครองและผู้สอน

ตารางเรียนที่แนะนำ

รูปแบบ	ระยะเวลา
สัปดาห์ละ 2 ครั้ง ครั้งละ 1–1.5 ชม. (1 บท/ครั้ง)	ประมาณ 13 สัปดาห์ (3 เดือน)
สัปดาห์ละ 1 ครั้ง	ประมาณ 26 สัปดาห์
ค่ายเข้มข้น วันละ 2 บท	ประมาณ 2–3 สัปดาห์

แนวทางสอน

- ให้เด็กพิมพ์เอง ผู้ใหญ่ทำหน้าที่ถามนำ เช่น "คิดว่าบรรทัดนี้ทำอะไร" มากกว่าบอกคำตอบ
- เมื่อติดบัก ให้ใช้ 5 ชั้นแก้บัก (บทที่ 25) ก่อนช่วยแก้ ปล่อยให้ติดสััก 10 นาทีก่อนช่วยเป็นเวลาที่ยืนยันรู้ดีที่สุด
- ชื่นชม กระบวนการ (ความพยายาม วิธีคิด) มากกว่าผลลัพธ์
- บทที่ 12 (IndexedDB/async) และบทที่ 22 (Cloud Sync) ยากที่สุด ให้เผื่อเวลาเพิ่มหรือแบ่งเป็น 2 ครั้ง ถ้าเด็กยังไม่พร้อมข้ามบทที่ 22 ไปก่อนได้โดยไม่กระทบบทอื่น (กลับมาทำภายหลัง)
- บทที่ 12 ใช้ได้เลยโดยไม่ต้องเข้าใจ Promise ลึก ๆ ให้เน้นใช้งานก่อน

เรื่องความปลอดภัยของเด็ก

- ใช้ ข้อมูลสมมติ เท่านั้น ไม่ใส่ข้อมูลลูกค้าจริงหรือข้อมูลส่วนบุคคลของใคร
- บัญชี GitHub, Firebase และบริการคลาวด์อื่น ผู้ปกครองควรเป็นผู้สร้างและดูแล โดยเฉพาะที่ผูกกับบัตรเครดิต (มีแผนฟรีเพียงพอสำหรับการเรียน)
- ไฟล์ที่มีรหัสลับ/credentials ต้องไม่ถูกอัปโหลดขึ้นที่สาธารณะ ฝึกใช้ .gitignore ตั้งแต่วัน
- ถ้าจะเปิดโค้ดจริงของธุรกิจให้เด็กดู ควรเป็นสำเนา ที่ลบรหัสลับและข้อมูลลูกค้าจริงออกแล้ว

ประเมินผลอย่างไร

ดูตาราง "โปรเจกต์จบหลักสูตร" ในหน้าถัดไป ให้คะแนนเป็นรายหัวข้อ และให้เด็กนำเสนอแอปของตัวเอง 5 นาที (อธิบายว่าทำอะไร ยากตรงไหน แก้อย่างไร)

โปรเจกต์จบหลักสูตร: "แอปร้านของฉัน"

เลือกธุรกิจสมมติที่ชอบ (ร้านขนม ร้านเช่าหนังสือ ร้านซ่อมจักรยาน ฯลฯ) แล้วสร้างแอปที่ใช้งานได้จริง โดยนำสิ่งที่เรียนมารวมกัน

หัวข้อ	สิ่งที่ต้องมี	คะแนน
หน้าตาและมือถือ	ใช้ CSS ร่วมกันทุกหน้า มีเมนู ใช้งานบนมือถือได้ (บท 4–5, 10)	10
ลูกค้าและสินค้า	เพิ่ม/แก้ไข/ซ่อน/ค้นหาได้ ข้อมูลไม่หายหลังรีเฟรช (บท 11–14)	20
ใบแจ้งหนี้	เลือกลูกค้า เพิ่มสินค้า คำนวณ VAT ออกเลขที่ไม่ซ้ำ กันบันทึกซ้ำ พิมพ์ได้ (บท 15–16)	20
การชำระเงิน	เก็บเงินเป็นสตางค์ รับชำระบางส่วน แสดงยอดค้างและสถานะถูกต้อง (บท 17)	15
รายงาน	แดชบอร์ดสรุปยอดและกราฟอย่างน้อย 1 อย่าง (บท 18)	10
สำรอง/นำเข้า-ส่งออก	ส่งออก CSV และสำรอง-กู้คืนข้อมูลได้ (บท 19, 23)	10
PWA	ติดตั้งและเปิดแบบออฟไลน์ได้ (บท 21)	5
เทสต์และ Git	มีเทสต์อย่างน้อย 5 ข้อและมีประวัติ commit อย่างน้อย 10 ครั้ง (บท 24–25)	5
นำเสนอ	อธิบายแอปและปัญหาที่เจอ 5 นาที	5
รวม		100

เหรียญรางวัลพิเศษ (ไม่บังคับ)

ล็อกอินพร้อมสิทธิ์ผู้ใช้ (บท 20) · ชิงก์ขึ้นคลาวด์ (บท 22) · ใบลดหนี้/คืนสินค้า (บท 26)

ภาคผนวก จ: เฉลยคำถามทบทวน

บท	คำถาม	เฉลย
1	HTML, CSS และ JavaScript ต่างกันอย่างไร (อธิบายสั้น ๆ)?	HTML = โครงสร้าง/เนื้อหา, CSS = หน้าตา, JavaScript = พฤติกรรมและตรรกะ
2	ถ้าลืมแท็กปิด เช่น ลืม <code></p></code> จะเกิดอะไรขึ้น?	Browser จะพยายามเดาและมักแสดงผลเพี้ยน หรือเนื้อหาต่อ ๆ ไปถูกจัดกลุ่มผิด จึงควรปิดแท็กเสมอ
3	ทำไม input แต่ละช่องควรมี id?	เพื่อให้ JavaScript และ label เรียกหาช่องนั้นได้อย่างถูกต้อง
4	ข้อดีของการใช้ตัวแปร CSS เช่น <code>--main</code> คืออะไร?	เปลี่ยนค่าที่เดียว สีทั้งเว็บเปลี่ยนตาม ดูง่ายและสม่ำเสมอ
5	Flexbox กับ Grid ใช้ต่างกันอย่างไร?	Flexbox จัดเรียงทีละแถว/ทีละคอลัมน์ ส่วน Grid จัดเป็นตารางสองมิติ
6	let กับ const ต่างกันอย่างไร?	let เปลี่ยนค่าได้ภายหลัง const กำหนดแล้วเปลี่ยนไม่ได้ (ใช้ const เป็นค่าเริ่มต้นดีที่สุด)
7	<code>[1,2,3].map(x => x * 2)</code> ได้อะไร?	<code>[2, 4, 6]</code>
8	ทำไมจึงต้องเรียก <code>event.preventDefault()</code> ตอน submit?	เพราะปกติฟอร์มจะรีโหลดหน้าเว็บ ซึ่งทำให้ข้อมูลใน JavaScript หายหมด
9	ทำไมจึงเก็บข้อมูลไว้ใน array แล้วค่อย <code>render()</code> แทนที่จะแก้ตารางตรง ๆ?	แยก "ข้อมูล" ออกจาก "หน้าจอ" ทำให้แก้ง่าย ไม่สับสน และต่อ ยอดไปบันทึกข้อมูลได้
10	ข้อดีของการแยก <code>nav.js</code> ออกมาคืออะไร?	แก้เมนูทีเดียว ทุกหน้าเปลี่ยนตาม ไม่ต้องแก้ซ้ำหลายไฟล์
11	ทำไมต้องครอบ <code>JSON.parse</code> ด้วย <code>try/catch</code> ?	ถ้าข้อมูลที่เก็บไว้เสียหาย <code>JSON.parse</code> จะ error และแอปจะพังทั้งหน้า
12	IndexedDB ดีกว่า <code>localStorage</code> อย่างไร (บอก 2 ข้อ)?	เก็บได้มากกว่า, เก็บ object ได้ตรง ๆ, ค้นหาด้วย index ได้เร็ว, มี transaction ป้องกันข้อมูลเสียหาย
13	CRUD ย่อมาจากอะไร?	Create, Read, Update, Delete
14	ทำไมใบแจ้งหนี้ต้องเก็บราคา ณ วันที่ออกไว้ในตัวเอง?	เพื่อให้ยอดในอดีตไม่เปลี่ยนเมื่อมีการปรับราคาสินค้าภายหลัง
15	<code>padStart(4, "0")</code> ทำอะไร?	เติมเลข 0 ข้างหน้าจนครบ 4 ตัว เช่น 7 กลายเป็น 0007
16	<code>@media print</code> ใช้ทำอะไร?	กำหนดหน้าตาเฉพาะตอนพิมพ์ เช่น ซ่อนปุ่มและเมนู
17	ทำไมจึงเก็บเงินเป็นสตางค์แทนบาทที่มีทศนิยม?	เพราะทศนิยมของคอมพิวเตอร์คลาดเคลื่อนได้ (เช่น <code>0.1+0.2</code>) จำนวนเต็มแม่นยำเสมอ
18	ขั้นตอนทำรายการ 'ยอดรวมต่อกลุ่ม' คืออะไร?	วนทุกรายการ ใช้ key ของกลุ่มเป็นกุญแจใน object แล้วบวกยอดสะสม
19	ทำไมต้องใส่ <code>\ufeff</code> (BOM) หน้าไฟล์ CSV ภาษาไทย?	ช่วยให้ Excel รู้ว่าไฟล์เป็น UTF-8 ภาษาไทยจะไม่กลายเป็นตัวอักษรเพี้ยน

บท	คำถาม	เฉลย
20	ทำไมไม่ควรเก็บรหัสผ่านตรง ๆ ในฐานข้อมูล?	ถ้าข้อมูลรั่ว ใครก็อ่านรหัสผ่านได้ จึงเก็บเฉพาะ hash ที่ย้อนกลับไม่ได้
21	Service Worker ทำอะไร?	ทำงานเบื้องหลัง ดักคำขอไฟล์และเก็บไฟล์ไว้ใน cache เพื่อให้ใช้ออฟไลน์ได้
22	ทำไมต้องมี updatedAt ในข้อมูลทุกชิ้น?	เพื่อเปรียบเทียบว่าฝั่งไหนใหม่กว่าตอน merge และเพื่อซิงค์เฉพาะที่เปลี่ยน
23	กฎ 3-2-1 ของการสำรองข้อมูลคืออะไร?	เก็บ 3 ชุด, ใน 2 สื่อแตกต่างกัน, และ 1 ชุดอยู่นอกสถานที่/บนคลาวด์
24	ทำไมควรเขียนเทสต์หลังเจอบั๊ก?	เพื่อกันไม่ให้บั๊กเดิมกลับมาอีกเมื่อมีการแก้ไขโค้ดในอนาคต
25	commit คืออะไร?	การบันทึกภาพรวมสถานะของโปรเจกต์ ณ เวลานั้น พร้อมข้อความอธิบาย ย้อนกลับมาดูได้ภายหลัง
26	PWA กับแอป Tauri ต่างกันอย่างไร?	PWA ทำงานในเบราว์เซอร์และติดตั้งจากเว็บ ส่วน Tauri ห่อเว็บเป็นโปรแกรมบนเครื่องที่เข้าถึงไฟล์ได้มากกว่า

ภารกิจท้าทายไม่มีเฉลยตายตัว ให้ผู้สอนดูว่าโปรแกรมทำงานตามโจทย์และเด็กอธิบายวิธีคิดได้

ภาคผนวก จ: ขั้นตอนวินิจฉัยปัญหา (Diagnosis Procedure)

เมื่อโปรแกรมมีปัญหา อย่าเดาสุ่มแก้ ให้ทำตามลำดับด้านล่างเหมือนหมอตรวจคนไข้: ชักอาการ → ตรวจ → ตั้งสมมติฐาน → ทดสอบ → ยืนยัน

ขั้นตอนหลัก 6 ข้อ (ใช้ได้กับทุกปัญหา)

ขั้น	ทำอะไร	คำถามที่ต้องตอบ
D0 หยุดและจด	หยุดแก้ จดอาการและสิ่งที่เปลี่ยนล่าสุด (ถ้ามี Git ให้รัน <code>git diff</code>)	เกิดอะไรขึ้น? เริ่มเป็นตอนไหน? แก้อะไรไปก่อนหน้านี้?
D1 ดู Console	กด F12 → แท็บ Console	มีข้อความ สีแดง ไหม? ไฟล์และบรรทัดไหน?
D2 จัดกลุ่มอาการ	เลือกอาการจากฝั่งข้อถัดไป แล้วตรวจตามที่ระบุ	ปัญหายูที่หน้าจอ ปุ่ม ข้อมูล หรือ การเก็บข้อมูล?
D3 ตัดให้เล็กที่สุด	คอมเมนต์โค้ดทั้งครึ่งหนึ่ง แล้วดูว่ายังพังไหม ทำซ้ำจนเหลือส่วนที่ทำให้พัง	บรรทัดไหนคือต้นเหตุ?
D4 สมมติฐานที่ละข้อ	ตั้งสมมติฐาน 1 ข้อ แก้ 1 อย่าง ทดสอบ ถ้าไม่ใช่ให้ย้อนกลับ (<code>git restore</code>) แล้วลองข้อถัดไป	แก้อย่างเดียวแล้วหายไหม?
D5 ยืนยันและกันกลับมา	ทดสอบซ้ำ 3 รอบ เขียนเทสต์ (บท 24) commit และจดลงบันทึกการแก้ปัญหา	ทำไมมันพัง? ครั้งหน้าจะป้องกันอย่างไร?

กฎหยุดและขอความช่วยเหลือ

ถ้าลองครบ 3 สมมติฐาน หรือติดนาน 30 นาทีแล้วยังไม่หาย ให้หยุด กรอก "การ์ดรายงานบั๊ก" (หน้าท้ายของภาคผนวกนี้) แล้วไปถามผู้สอน การติดนานเกินไปไม่ได้ทำให้เก่งขึ้น

D2: ผังวินิจฉัยตามอาการ

เริ่มจากอาการที่เห็น แล้วตรวจตามลำดับ จากซ้ายไปขวา ข้อแรกที่ใช้คือสาเหตุที่น่าจะเป็นมากที่สุด

อาการ	ตรวจตามลำดับนี้	วิธีแก้
A. หน้าเว็บขาว/ไม่ขึ้นอะไร	1) Console มีแดงไหม 2) Network: ไฟล์ HTML/JS/CSS เป็น 200 ไหม (404 = ผิดชื่อ/ที่อยู่) 3) เปิดผ่าน Live Server หรือ file://	แก้ชื่อไฟล์/path (ตัวพิมพ์เล็ก-ใหญ่มีผล), เปิดด้วย Live Server
B. หน้าตาไม่ตรงที่เขียน	1) ลืม <link> เชื่อม CSS? 2) คลิกขวา Inspect ดู Styles ถูกขีดฆ่าไหม (กฎอื่นทับ) 3) แคชค้าง? ลอง Ctrl+Shift+R 4) Service Worker เก็บของเก่า?	เชื่อมต่อไฟล์ให้ถูก, ใช้ตัวเลือกที่เจาะจงกว่า, ล้างแคช/เปลี่ยนเลข CACHE
C. กดปุ่มแล้วไม่เกิดอะไร	1) ใส่ <code>console.log("clicked")</code> ใน handler ขึ้นใหม่ 2) <code>getElementById</code> คืน null ไหม (id ตรงไหม) 3) script อยู่ก่อน HTML ที่มันเรียกหรือเปล่า 4) ฟังก์ชันรีโหนดหน้า? ลืม <code>preventDefault()</code>	แก้ id, ย้าย script ไปท้าย body, เพิ่ม <code>preventDefault</code>
D. ตัวเลข/ผลลัพธ์ผิด	1) <code>console.log</code> ค่าก่อนและหลังบรรทัดคำนวณ 2) <code>typeof</code> เป็น string ไหม (<code>"5"+"5"</code> ได้ <code>"55"</code>) 3) เป็นเรื่องทศนิยมเงินไหม (0.1+0.2) 4) ตัวกรองหรือสถานะ cancelled ตัดรายการออกไหม	แปลงด้วย <code>Number()</code> , เก็บเงินเป็นสตางค์ (บท 17), ตรวจสอบเงื่อนไข filter
E. ข้อมูลหายหลังรีเฟรช	1) F12 → Application → IndexedDB/Local Storage ข้อมูลอยู่ไหม 2) ถ้าไม่อยู่: บันทึกรจริงไหม ลืม <code>await</code> /error เจียบ? 3) ถ้าอยู่แต่ไม่แสดง: ตัวค้นหา/ตัวกรอง/ธง deleted ซ่อนไว้?	เติม <code>await</code> + <code>try/catch</code> , ล้างตัวกรอง, ตรวจสอบฟิลด์ <code>deleted/cancelled</code>
F. ข้อมูลหายเมื่อสลับเครื่อง	1) ซิงค์ถูกเปิดใช้ไหม ออนไลน์ไหม 2) <code>updatedAt</code> ถูกใส่ทุกครั้งที่แก้ไหม 3) ลบแล้ว "ฟื้นคืนชีพ"? ใช้ tombstone หรือยัง	ใส่ <code>updatedAt</code> , ใช้ธง <code>deleted</code> , ตรวจสอบกฎ Firestore และโควตา
G. ใช้ออฟไลน์ไม่ได้ / แอปไม่อัปเดต	1) Application → Manifest มี error ไหม 2) Service Worker สถานะ activated ไหม 3) เลข CACHE ถูกเปลี่ยนหลังแก้ไฟล์ไหม 4) ไฟล์ใน FILES ครบไหม	เปลี่ยน v1 → v2, ดึง Update on reload ตอนพัฒนา, เติมไฟล์ที่ขาด
H. ล็อกอินไม่ได้ / สิทธิ์ผิด	1) hash ของรหัสผ่านที่เก็บกับที่คำนวณตรงกันไหม (เก็บด้วยวิธีเดียวกัน?) 2) role สอดคล้องกับ PERMISSIONS ไหม 3) เปิดผ่าน https/localhost ไหม (crypto.subtle ต้องการ)	แก้การสะกด, ใช้ฟังก์ชัน hash เดียวกันทั้งตอนสมัครและล็อกอิน

ตารางถอดรหัสข้อความ Error

ข้อความที่เห็นใน Console	แปลว่า	ตรวจอะไรก่อน
SyntaxError: Unexpected token	เขียนผิดไวยากรณ์ โปรแกรมไม่ทันได้รันเลย	วงเล็บ {{ }} () เครื่องหมายคำพูด จุลภาค ที่ขาด/เกิน ไกล่บรรทัดที่ระบุ
ReferenceError: x is not defined	ใช้ชื่อที่ยังไม่มีอยู่	สะกดผิด? ยังไม่ประกาศ? ไฟล์ที่ประกาศถูกโหลดทีหลัง?
TypeError: Cannot read properties of null	หาค่าประกอบ/ข้อมูลไม่เจอแล้วไปใช้ต่อ	id ถูกไหม script โหลดก่อน HTML ไหม; ใช้ ?. และเช็คก่อนใช้
TypeError: x is not a function	เรียกสิ่งที่ไม่ใช่ฟังก์ชัน	ชื่อตัวแปรชนกับชื่อฟังก์ชัน? ลืมโหลดไฟล์?
Failed to load resource: 404	หาไฟล์ไม่เจอ	ชื่อไฟล์ ที่อยู่ ตัวพิมพ์เล็กใหญ่
Uncaught (in promise) ...	error ในโค้ดที่ใช้ await/Promise ไม่มีใครดักจับ	ครอบ try/catch รอบ await; อ่านข้อความด้านล่างบรรทัดนี้
QuotaExceededError	พื้นที่เก็บข้อมูลเต็ม (มักเป็น localStorage)	ย้ายข้อมูลใหญ่ไป IndexedDB (บท 12)
CORS / blocked by policy	เรียกข้อมูลข้ามโดเมนไม่ได้รับอนุญาต	เปิดผ่าน file:// ไหม? ใช้ Live Server; ตรวจกฎฝั่งเซิร์ฟเวอร์
permission-denied (Firebase)	กฎความปลอดภัยไม่อนุญาต	ล็อกอินหรือยัง? ตรวจ firestore.rules

สคริปต์ตรวจสอบสุขภาพแอป (Health Check)

วางสคริปต์นี้ใน Console (F12 → Console) ของหน้าที่มีปัญหา แล้วกด Enter มันจะตรวจ 10 เรื่องพื้นฐานและบอกคำแนะนำให้เองในรูปแบบตาราง ใช้ก่อนเริ่มขั้น D2 เพื่อตัดปัญหาพื้นฐานออกไปก่อน

health-check.js (วางใน Console)

```
(async () => {
  const r = [];
  const chk = (name, pass, hint) => r.push({ "ตรวจ": name, "ผล": pass ? "ผ่าน" : "ไม่ผ่าน", "คำแนะนำ": pass ? "" : hint });
  chk("เปิดผ่าน localhost/https (ไม่ใช่ file://)", location.protocol !== "file:", "เปิดด้วย Live Server");
  chk("มี meta charset UTF-8", !!document.querySelector("meta[charset]"), "เพิ่ม <meta charset='UTF-8'> ใน head");
  chk("มี meta viewport", !!document.querySelector("meta[name=viewport]"), "เพิ่ม meta viewport เพื่อให้ใช้บนมือถือได้");
  const ids = [...document.querySelectorAll("[id]").map(e => e.id)];
  const dup = [...new Set(ids.filter((x, i) => ids.indexOf(x) !== i))];
  chk("id ในหน้าไม่ซ้ำกัน", dup.length === 0, "id ซ้ำ: " + dup.join(", "));
  const urls = [...document.scripts].map(s => s.src).concat([...document.querySelectorAll("link[rel=stylesheet]").map(l => l.href)].filter(Boolean));
  for (const u of urls) {
    let okLoad = false;
    try { okLoad = (await fetch(u, { method: "HEAD" })).ok; } catch {}
    chk("โหลดไฟล์ได้: " + u.split("/").pop(), okLoad, "ชื่อไฟล์/ที่อยู่ผิด หรือไฟล์ไม่มีอยู่ (404)");
  }
  let ls = true; try { localStorage.setItem("_t", 1); localStorage.removeItem("_t"); } catch { ls = false; }
  chk("localStorage ใช้งานได้", ls, "ปิดโหมดส่วนตัว หรือตรวจการตั้งค่าเบราว์เซอร์");
  chk("เบราว์เซอร์รองรับ IndexedDB", "indexedDB" in window, "ใช้ Chrome เวอร์ชันใหม่");
  if (indexedDB.databases) {
    const dbs = await indexedDB.databases();
    chk("พบฐานข้อมูล IndexedDB (" + dbs.map(d => d.name).join(", ") + ")", dbs.length > 0, "ยังไม่เคยบันทึกข้อมูล หรือชื่อ DB ไม่ตรง");
  }
  if ("serviceWorker" in navigator) {
    const regs = await navigator.serviceWorker.getRegistrations();
    chk("Service Worker ลงทะเบียนแล้ว", regs.length > 0, "ยังไม่ได้ register('sw.js') (ถ้าไม่ได้ทำ PWA ช้ามได้)");
  }
  chk("ออนไลน์อยู่", navigator.onLine, "อินเทอร์เน็ตหลุด (พีเจอาร์ชิ่งจะไม่ทำงาน)");
  console.table(r);
  console.log("ข้อที่ไม่ผ่าน:", r.filter(x => x["ผล"] === "ไม่ผ่าน").length);
})();
```

อย่าลืม

ถ้า Chrome ไม่ให้วาง ให้พิมพ์ `allow pasting` แล้วกด Enter ก่อน เป็นระบบป้องกันไม่ให้ใครลอกให้เราวางโค้ดแปลก ๆ ดังนั้น วางเฉพาะโค้ดที่เราอ่านเข้าใจแล้วเท่านั้น

การร้ดรายงานบั๊ก และบันทึกการแก้บั๊ก

การร้ดรายงานบั๊ก (กรอกก่อนถามผู้สอน)

หัวข้อ	กรอก
1. หน้า/ไฟล์ที่มีปัญหา	
2. สิ่งที อยากรให้ เกิดขึ้น	
3. สิ่งที เกิดขึ้นจริง	
4. ขั้นตอนทำให้เกิดซ้ำ (1, 2, 3...)	
5. ข้อความ error ใน Console (คัดลอกทั้งบรรทัด)	
6. เปลี่ยนอะไรไปก่อนที่จะพัง	
7. ลองแก้อะไรไปแล้วบ้าง (อย่างน้อย 3 ข้อ)	

บันทึกการแก้บั๊ก (Diagnosis Log)

วันที่	อาการ	สาเหตุที่แท้จริง	วิธีแก้	จะป้องกันอย่างไร

ทบทวนบันทึกนี้ทุกเดือน บั๊กทีเจอซ้ำ ๆ คือจุดทีควรเขียนเทสต์หรือปรับวิธีทำงาน

ประกาศนียบัตร

นักสร้างแอปรุ่นเยาว์

ขอมอบให้

ผู้สร้างแอปจัดการใบแจ้งหนี้สำเร็จตั้งแต่ศูนย์
ด้วยความมุ่งมั่น ความพยายาม และไม่ยอมแพ้ต่อบั๊กใด ๆ

วันที่ ลงชื่อผู้สอน